异步图书
www.epubit.com.cn

从Excel到Python

数据分析进阶指南

王彦平（蓝鲸）／ 著



中国工信出版集团
人民邮电出版社
POSTS & TELECOM PRESS

异步社区会员 lizhimiao(18926120246) 专享 尊重版权

目　　录

版权信息

作者简介

序

第1章 生成数据表

第2章 数据表检查

第3章 数据表清洗

第4章 数据预处理

第5章 数据提取

第6章 数据筛选

第7章 数据汇总

第8章 数据统计

第9章 数据输出

案例 990万次骑行：纽约自行车共享系统分析

结束语

异步社区会员 lizhimiao(18926120246) 专享 尊重版权

版权信息

书名：从Excel到Python——数据分析进阶指南

本书由人民邮电出版社发行数字版。版权所有，侵权必究。

您购买的人民邮电出版社电子书仅供您个人使用，未经授权，不得以任何方式复制和传播本书内容。

我们愿意相信读者具有这样的良知和觉悟，与我们共同保护知识产权。

如果购买者有侵权行为，我们可能对该用户实施包括但不限于关闭该帐号等维权措施，并可能追究法律责任。

异步社区会员 lizhimiao(18926120246) 专享 尊重版权

作者简介

王彦平 (蓝鲸)，Google分析个人资格认证，专注于网站数据分析实践及Google Analytics应用研究。创建“蓝鲸的网站分析笔记”博客分享网站分析经验与技巧，被Avinash Kaushia先生推荐为探索Google Analytics必读的中文博客。同时王彦平还是艾瑞网专家，艾瑞商学院讲师，Digital Analytics Association会员和互联网著名开放式分类目录（Open Directory Project）DMOZ网站的志愿编辑。

王彦平推崇以用户体验为中心的网站数据分析实践，阐释了目标、用户、体验三位一体的网站分析体系。相信GIGO（Garbage In Garbage Out）原则是帮助分析师发现数据价值的关键要素，网站追踪代码设计则是确保业务与数据彼此相连，赋予数据灵魂的重要环节。

王彦平是《网站分析实战—如何以数据驱动决策，提升网站价值》《人人都是网站分析师—从分析师的视角理解网站和解读数据》两本书的作者，及《流量的秘密—Google Analytics网站分析与优化技巧》（第2版）的译者。其中面向初学者的《人人都是网站分析师》一书被Brian Clifton博士(*Google Web Analytics*欧洲负责人及 *Google Analytics Individual Qualification* 创建者)推荐为新人入门必备书籍。

王彦平拥有通信、地产、汽车和零售等多个行业的网站数据分析及优化经验，服务的客户包括中国联通、链家地产、摩点众筹、克莱

异步社区会员 lizhimiao(18926120246) 专享 尊重版权

欢迎加入非盈利Python编程学习交流QQ群783462347，群里免费提供500+本Python书籍！

斯勒、一汽丰田、雷克萨斯、北京现代、海南航空、加多宝、玛萨玛索、中国日报、三精制药等。

电子邮件：cliff1980@gmail.com

新浪微博：@蓝鲸碎碎念

微信公众号：bluewhale_cc



异步社区会员 lizhimiao(18926120246) 专享 尊重版权

欢迎加入非盈利Python编程学习交流QQ群783462347，群里免费提供500+本Python书籍！

序

Excel是数据分析中最常用的工具，本书通过Python与Excel的功能对比介绍如何使用Python通过函数式编程完成Excel中的数据处理及分析工作。在Python中pandas库用于数据处理，我们从1787页的pandas官网文档中总结出最常用的36个函数，通过这些函数介绍如何通过Python完成数据生成和导入、数据清洗、预处理，以及最常见的数据分类，数据筛选，分类汇总，透视等最常见的操作。

异步社区会员 lizhimiao(18926120246) 专享 尊重版权

欢迎加入非盈利Python编程学习交流QQ群783462347，群里免费提供500+本Python书籍！

第1章 生成数据表

常见的生成数据表的方法有两种，第一种是导入外部数据，第二种是直接写入数据。 Excel中的“文件”菜单中提供了获取外部数据的功能，支持数据库和文本文件和页面的多种数据源导入。



Python支持从多种类型的数据导入。在开始使用Python进行数据导入前需要先导入pandas库，为了方便起见，我们也同时导入numpy库。

```
import numpy as np
import pandas as pd
```

1. 导入数据表

下面分别是从Excel和csv格式文件中导入数据并创建数据表的方法。代码是最简模式，里面有很多可选参数设置，例如列名称、索引列、数据格式等。感兴趣的朋友可以参考pandas的官方文档。

异步社区会员 lizhimiao(18926120246) 专享 尊重版权

欢迎加入非盈利Python编程学习交流QQ群783462347，群里免费提供500+本Python书籍！

```
df=pd.DataFrame(pd.read_csv('name.csv',header=1))
df=pd.DataFrame(pd.read_Excel('name.xlsx'))
```

2. 创建数据表

另一种方法是通过直接写入数据来生成数据表，Excel中直接在单元格中输入数据就可以，Python中通过下面的代码来实现。生成数据表的函数是pandas库中的DateFrame函数，数据表一共有6行数据，每行有6个字段。在数据中我们特意设置了一些NA值和有问题的字段，例如包含空格等。后面将在数据清洗步骤进行处理。后面我们将统一以DataFrame的简称df来命名数据表。

```
df = pd.DataFrame({"id":[1001,1002,1003,1004,1005,1006],
                   "date":pd.date_range('20130102', periods=6),
                   "city":['Beijing ', 'SH', ' guangzhou ', 'Shen
zhen', 'shanghai', 'BEIJING '],
                   "age":[23,44,54,32,34,32],
                   "category":['100-A','100-B','110-A','110-C','2
10-A','130-F'],
                   "price":[1200,np.nan,2133,5433,np.nan,4432]},
                   columns =['id','date','city','category','age',
'price'])
```

这是刚刚创建的数据表，我们没有设置索引列，price字段中包含有NA值，city字段中还包含了一些脏数据。

异步社区会员 lizhimiao(18926120246) 专享 尊重版权

欢迎加入非盈利Python编程学习交流QQ群783462347，群里免费提供500+本Python书籍！

	id	date	city	category	age	price
0	1001	2013-01-02	Beijing	100-A	23	1200.0
1	1002	2013-01-03	SH	100-B	44	NaN
2	1003	2013-01-04	guangzhou	110-A	54	2133.0
3	1004	2013-01-05	Shenzhen	110-C	32	5433.0
4	1005	2013-01-06	shanghai	210-A	34	NaN
5	1006	2013-01-07	BEIJING	130-F	32	4432.0

异步社区会员 lizhimiao(18926120246) 专享 尊重版权

欢迎加入非盈利Python编程学习交流QQ群783462347，群里免费提供500+本Python书籍！

第2章 数据表检查

本章主要介绍对数据表进行检查。Python中处理的数据量通常会比较大，比如纽约的出租车数据和Citibike的骑行数据，其数据量都在千万级，我们无法一目了然地了解数据表的整体情况，必须要通过一些方法来获得数据表的关键信息。数据表检查的另一个目的是了解数据的概况，例如整个数据表的大小、所占空间、数据格式、是否有空值和重复项和具体的数据内容，为后面的清洗和预处理做好准备。

1. 数据维度(行列)

Excel中可以通过CTRL+向下的光标键，和CTRL+向右的光标键来查看行号和列号。Python中使用shape函数来查看数据表的维度，也就是行数和列数，函数返回的结果(6,6)表示数据表有6行，6列。下面是具体的代码。

```
#查看数据表的维度
df.shape
(6, 6)
```

2. 数据表信息

使用info函数查看数据表的整体信息，这里返回的信息比较多，包括数据维度、列名称、数据格式和所占空间等信息。

异步社区会员 lizhimiao(18926120246) 专享 尊重版权

欢迎加入非盈利Python编程学习交流QQ群783462347，群里免费提供500+本Python书籍！

```
#数据表信息
df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6 entries, 0 to 5
Data columns (total 6 columns):
id          6 non-null int64
date        6 non-null datetime64[ns]
city        6 non-null object
category    6 non-null object
age         6 non-null int64
price       4 non-null float64
dtypes: datetime64[ns](1), float64(1), int64(2), object(2)
memory usage: 368.0+ bytes
```

3. 查看数据格式

Excel中通过选中单元格并查看开始菜单中的数值类型来判断数据的格式。Python中使用dtypes函数来返回数据格式。



Dtypes是一个查看数据格式的函数，可以一次性查看数据表中所有数据的格式，也可以指定一列来单独查看。

```
#查看数据表各列格式
df.dtypes
id                   int64
date        datetime64[ns]
city                object
category            object
age                  int64
price              float64
dtype: object
```


异步社区会员 lizhimiao(18926120246) 专享 尊重版权

```
#查看单列格式
df['B'].dtype
dtype('int64')
```

4. 查看空值

Excel中查看空值的方法是使用“定位条件”功能对数据表中的空值进行定位。“定位条件”在“开始”目录下的“查找和选择”目录中。



Isnull是Python中检验空值的函数，返回的结果是逻辑值，包含空值返回True，不包含则返回False。用户既可以对整个数据表进行检查，也可以单独对某一列进行空值检查。

```
#检查数据空值
```


异步社区会员 lizhimiao(18926120246) 专享 尊重版权

```
df.isnull()
```

	id	date	city	category	age	price
0	False	False	False	False	False	False
1	False	False	False	False	False	True
2	False	False	False	False	False	False
3	False	False	False	False	False	False
4	False	False	False	False	False	True
5	False	False	False	False	False	False

```
#检查特定列空值
df['price'].isnull()

0    False
1     True
2    False
3    False
4     True
5    False
Name: price, dtype: bool
```

5. 查看唯一值

Excel中查看唯一值的方法是使用“条件格式”对唯一值进行颜色标记。Python中使用unique函数查看唯一值。



Unique是查看唯一值的函数，只能对数据表中的特定列进行检

异步社区会员 lizhimiao(18926120246) 专享 尊重版权

查。下面是代码，返回的结果是该列中的唯一值。类似与Excel中删除重复项后的结果。

```
#查看city列中的唯一值
df['city'].unique()

array(['Beijing ', 'SH', ' guangzhou ', 'Shenzhen', 'shanghai', '
BEIJING '], dtype=object)
```

6. 查看数据表数值

Python中的Values函数用来查看数据表中的数值。以数组的形式返回，不包含表头信息。

```
#查看数据表的值
df.values

array([[1001, Timestamp('2013-01-02 00:00:00'), 'Beijing ', '100-
A', 23,
        1200.0],
       [1002, Timestamp('2013-01-03 00:00:00'), 'SH', '100-B', 44
, nan],
       [1003, Timestamp('2013-01-04 00:00:00'), ' guangzhou ', '1
10-A', 54,
        2133.0],
       [1004, Timestamp('2013-01-05 00:00:00'), 'Shenzhen', '110-
C', 32,
        5433.0],
       [1005, Timestamp('2013-01-06 00:00:00'), 'shanghai', '210-
A', 34,
        nan],
       [1006, Timestamp('2013-01-07 00:00:00'), 'BEIJING ', '130-
F', 32,
        4432.0]], dtype=object)
```

7. 查看列名称

异步社区会员 lizhimiao(18926120246) 专享 尊重版权

Colums函数用来单独查看数据表中的列名称。

```
#查看列名称
df.columns

Index(['id', 'date', 'city', 'category', 'age', 'price'], dtype='
object')
```

8. 查看前10行数据

Head函数用来查看数据表中的前N行数据，默认head()显示前10行数据，可以自己设置参数值来确定查看的行数。下面的代码中设置查看前3行的数据。

```
#查看前3行数据
df.head(3)
```

	id	date	city	category	age	price
0	1001	2013-01-02	Beijing	100-A	23	1200.0
1	1002	2013-01-03	SH	100-B	44	NaN
2	1003	2013-01-04	guangzhou	110-A	54	2133.0

9. 查看后10行数据

Tail行数与head函数相反，用来查看数据表中后N行的数据，默认tail()显示后10行数据，可以自己设置参数值来确定查看的行数。下面的代码中设置查看后3行的数据。

```
#查看最后3行
df.tail(3)
```


异步社区会员 lizhimiao(18926120246) 专享 尊重版权

欢迎加入非盈利Python编程学习交流QQ群783462347，群里免费提供500+本Python书籍！

	id	date	city	category	age	price
3	1004	2013-01-05	Shenzhen	110-C	32	5433.0
4	1005	2013-01-06	shanghai	210-A	34	NaN
5	1006	2013-01-07	BEIJING	130-F	32	4432.0

异步社区会员 lizhimiao(18926120246) 专享 尊重版权

欢迎加入非盈利Python编程学习交流QQ群783462347，群里免费提供500+本Python书籍！

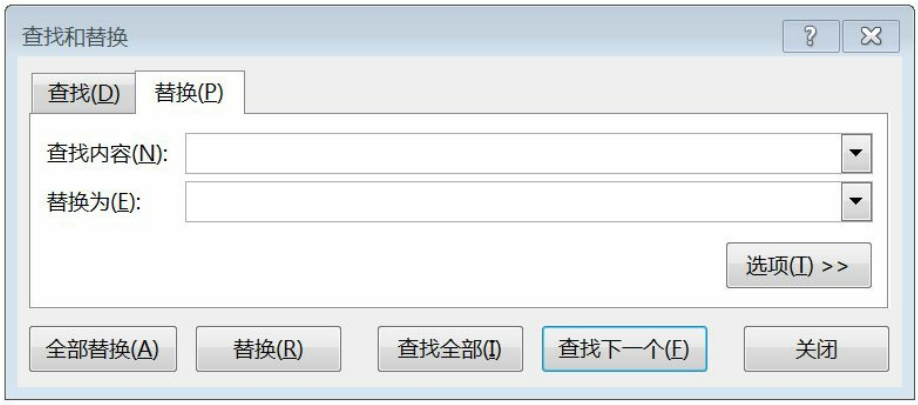
第3章 数据表清洗

本章介绍的是对数据表中的问题进行清洗，主要内容包括对空值、大小写问题、数据格式和重复值的处理。这里不包含对数据间的逻辑验证。

1. 处理空值(删除或填充)

我们在创建数据表的时候在price字段中故意设置了几个NA值。对于空值的处理方式有很多种，可以直接删除包含空值的数据，也可以对空值进行填充，比如用0填充或者用均值填充。还可以根据不同字段的逻辑对空值进行推算。

Excel中可以通过“查找和替换”功能对空值进行处理，将空值统一替换为0或均值。也可以通过“定位”空值来实现。



Python中处理空值的方法比较灵活，可以使用 Dropna函数用来删

异步社区会员 lizhimiao(18926120246) 专享 尊重版权

欢迎加入非盈利Python编程学习交流QQ群783462347，群里免费提供500+本Python书籍！

除数据表中包含空值的数据，也可以使用fillna函数对空值进行填充。下面的代码和结果中可以看到使用dropna函数后，包含NA值的两个字段已经不见了。返回的是一个不包含空值的数据表。

```
#删除数据表中含有空值的行
df.dropna(how='any')
```

	id	date	city	category	age	price
0	1001	2013-01-02	Beijing	100-A	23	1200.0
2	1003	2013-01-04	guangzhou	110-A	54	2133.0
3	1004	2013-01-05	Shenzhen	110-C	32	5433.0
5	1006	2013-01-07	BEIJING	130-F	32	4432.0

除此之外也可以使用数字对空值进行填充，下面的代码使用fillna函数对空值字段填充数字0。

```
#使用数字0填充数据表中空值
df.fillna(value=0)
```

我们选择填充的方式来处理空值，使用price列的均值来填充NA字段，同样使用fillna函数，在要填充的数值中使用mean函数先计算price列当前的均值，然后使用这个均值对NA进行填充。可以看到两个空值字段显示为3299.5

```
#使用price均值对NA进行填充
df['price'].fillna(df['price'].mean())

0    1200.0
1    3299.5
2    2133.0
3    5433.0
4    3299.5
```


异步社区会员 lizhimiao(18926120246) 专享 尊重版权

```
5    4432.0
Name: price, dtype: float64
```

	id	date	city	category	age	price
0	1001	2013-01-02	Beijing	100-A	23	1200.0
1	1002	2013-01-03	SH	100-B	44	3299.5
2	1003	2013-01-04	guangzhou	110-A	54	2133.0
3	1004	2013-01-05	Shenzhen	110-C	32	5433.0
4	1005	2013-01-06	shanghai	210-A	34	3299.5
5	1006	2013-01-07	BEIJING	130-F	32	4432.0

2. 清理空格

除了空值，字符中的空格也是数据清洗中一个常见的问题，下面是清除字符中空格的代码。

```
#清除city字段中的字符空格
df['city']=df['city'].map(str.strip)
```

3. 大小写转换

在英文字段中，字母的大小写不统一也是一个常见的问题。Excel中有UPPER，LOWER等函数，Python中也有同名函数用来解决大小写的问题。在数据表的city列中就存在这样的问题。我们将city列的所有字母转换为小写。下面是具体的代码和结果。

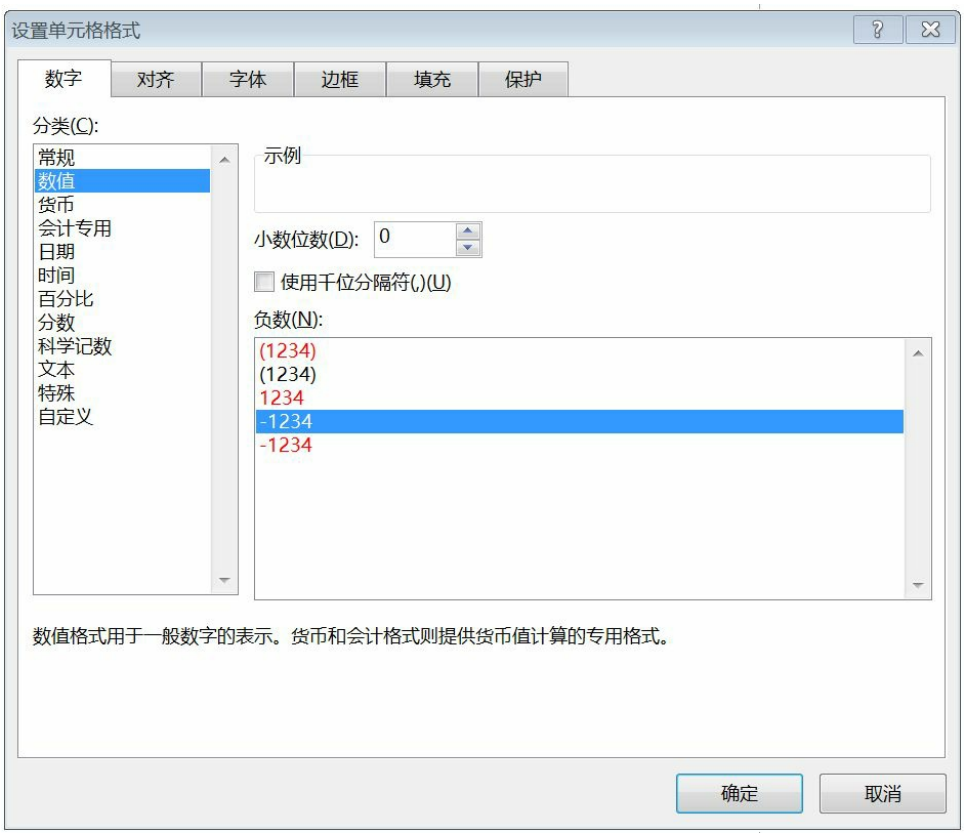
```
city列大小写转换
df['city']=df['city'].str.lower()
```


异步社区会员 lizhimiao(18926120246) 专享 尊重版权

	id	date	city	category	age	price
0	1001	2013-01-02	beijing	100-A	23	1200.0
1	1002	2013-01-03	sh	100-B	44	3299.5
2	1003	2013-01-04	guangzhou	110-A	54	2133.0
3	1004	2013-01-05	shenzhen	110-C	32	5433.0
4	1005	2013-01-06	shanghai	210-A	34	3299.5
5	1006	2013-01-07	beijing	130-F	32	4432.0

4. 更改数据格式

Excel中通过“设置单元格格式”功能可以修改数据格式。Python中通过astype函数用来修改数据格式。



异步社区会员 lizhimiao(18926120246) 专享 尊重版权

Python中dtype是查看数据格式的函数，与之对应的是astype函数，用来更改数据格式。下面的代码中将price字段的值修改为int格式。

```
#更改数据格式
df['price'].astype('int')

0    1200
1    3299
2    2133
3    5433
4    3299
5    4432
Name: price, dtype: int32
```

5. 更改列名称

Rename是更改列名称的函数，我们将来数据表中的category列更改为category-size。下面是具体的代码和更改后的结果。

```
#更改列名称
df.rename(columns={'category': 'category-size'})
```

	id	date	city	category-size	age	price
0	1001	2013-01-02	beijing	100-A	23	1200
1	1002	2013-01-03	sh	100-B	44	3299
2	1003	2013-01-04	guangzhou	110-A	54	2133
3	1004	2013-01-05	shenzhen	110-C	32	5433
4	1005	2013-01-06	shanghai	210-A	34	3299
5	1006	2013-01-07	beijing	130-F	32	4432

6. 删除重复值

异步社区会员 lizhimiao(18926120246) 专享 尊重版权

很多数据表中还包含重复值的问题，Excel的数据目录下有“删除重复项”的功能，可以用来删除数据表中的重复值。默认Excel会保留最先出现的数据，删除后面重复出现的数据。



Python中使用drop_duplicates函数删除重复值。我们以数据表中的city列为例，city字段中存在重复值。默认情况下drop_duplicates()将删除后出现的重复值(与Excel逻辑一致)。增加keep='last'参数后将删除最先出现的重复值，保留最后的值。下面是具体的代码和比较结果。

原始的city列中beijing存在重复，分别在第一位和最后一位。

```
df['city']
0      beijing
1           sh
2    guangzhou
3     shenzhen
4     shanghai
5      beijing
Name: city, dtype: object
```

使用默认的drop_duplicates()函数删除重复值，从结果中可以看到第一位的beijing被保留，最后出现的beijing被删除。

```
#删除后出现的重复值
df['city'].drop_duplicates()
0      beijing
1           sh
2    guangzhou
```


异步社区会员 lizhimiao(18926120246) 专享 尊重版权

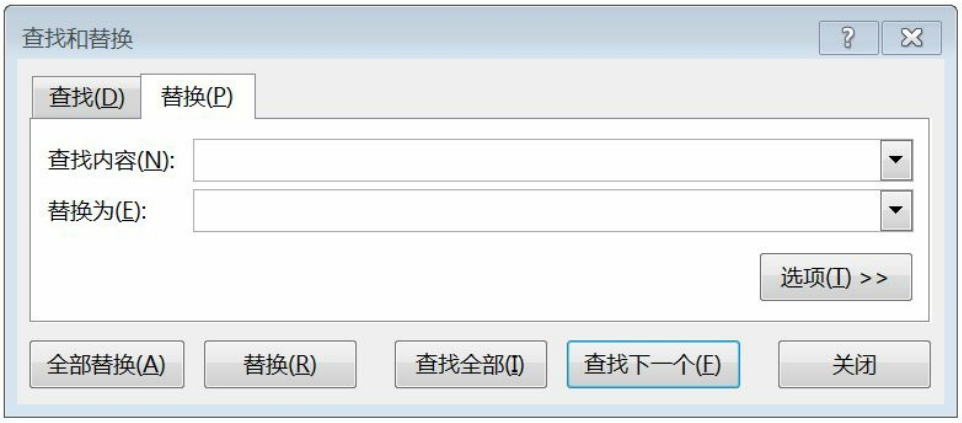
```
3     shenzhen
4     shanghai
Name: city, dtype: object
```

设置keep='last‘’参数后，与之前删除重复值的结果相反，第一位出现的beijing被删除，保留了最后一位出现的beijing。

```
#删除先出现的重复值
df['city'].drop_duplicates(keep='last')
1           sh
2    guangzhou
3     shenzhen
4     shanghai
5      beijing
Name: city, dtype: objec
```

7. 数值修改及替换

数据清洗中最后一个问题是数值修改或替换，Excel中使用“查找和替换”功能就可以实现数值的替换。



Python中使用replace函数实现数据替换。数据表中city字段上海存在两种写法，分别为shanghai和SH。我们使用replace函数对SH进行替换。

异步社区会员 lizhimiao(18926120246) 专享 尊重版权

欢迎加入非盈利Python编程学习交流QQ群783462347，群里免费提供500+本Python书籍！

```
#数据替换
df['city'].replace('sh', 'shanghai')
0      beijing
1     shanghai
2    guangzhou
3     shenzhen
4     shanghai
5      beijing
Name: city, dtype: object
```


异步社区会员 lizhimiao(18926120246) 专享 尊重版权

欢迎加入非盈利Python编程学习交流QQ群783462347，群里免费提供500+本Python书籍！

第4章 数据预处理

本章主要讲的是数据的预处理，对清洗完的数据进行整理以便后期的统计和分析工作。主要包括数据表的合并，排序，数值分列，数据分组及标记等工作。

1. 数据表合并

首先是对不同的数据表进行合并，我们这里创建一个新的数据表df1，并将df和df1两个数据表进行合并。在Excel中没有直接完成数据表合并的功能，可以通过VLOOKUP函数分步实现。在Python中可以通过merge函数一次性实现。下面建立df1数据表，用于和df数据表进行合并。

```
df1=pd.DataFrame({"id":[1001,1002,1003,1004,1005,1006,1007,1008],
                  "gender":['male','female','male','female','male
','female','male','female'],
                  "pay":['Y','N','Y','Y','N','Y','N','Y',],
                  "m-point":[10,12,20,40,40,40,30,20]})
```


异步社区会员 lizhimiao(18926120246) 专享 尊重版权

	gender	id	m-point	pay
0	male	1001	10	Y
1	female	1002	12	N
2	male	1003	20	Y
3	female	1004	40	Y
4	male	1005	40	N
5	female	1006	40	Y
6	male	1007	30	N
7	female	1008	20	Y

使用merge函数对两个数据表进行合并，合并的方式为inner，将两个数据表中共有的数据匹配到一起生成新的数据表。并命名为df_inner。

```
#数据表匹配合并
df_inner=pd.merge(df,df1,how='inner')
```

	id	date	city	category	age	price	gender	m-point	pay
0	1001	2013-01-02	beijing	100-A	23	1200	male	10	Y
1	1002	2013-01-03	shanghai	100-B	44	3299	female	12	N
2	1003	2013-01-04	guangzhou	110-A	54	2133	male	20	Y
3	1004	2013-01-05	shenzhen	110-C	32	5433	female	40	Y
4	1005	2013-01-06	shanghai	210-A	34	3299	male	40	N
5	1006	2013-01-07	beijing	130-F	32	4432	female	40	Y

除了inner方式以外，合并的方式还有left，right和outer方式。这几种方式的差别在我其他的文章中有详细的说明和对比。

```
df_left=pd.merge(df,df1,how='left')
df_right=pd.merge(df,df1,how='right')
df_outer=pd.merge(df,df1,how='outer')
```


异步社区会员 lizhimiao(18926120246) 专享 尊重版权

欢迎加入非盈利Python编程学习交流QQ群783462347，群里免费提供500+本Python书籍！

2. 设置索引列

完成数据表的合并后，我们对df_inner数据表设置索引列，索引列的功能很多，可以进行数据提取，汇总，也可以进行数据筛选等。设置索引的函数为set_index。

```
#设置索引列
df_inner.set_index('id')
```

	date	city	category	age	price	gender	m-point	pay
id								
1001	2013-01-02	beijing	100-A	23	1200	male	10	Y
1002	2013-01-03	shanghai	100-B	44	3299	female	12	N
1003	2013-01-04	guangzhou	110-A	54	2133	male	20	Y
1004	2013-01-05	shenzhen	110-C	32	5433	female	40	Y
1005	2013-01-06	shanghai	210-A	34	3299	male	40	N
1006	2013-01-07	beijing	130-F	32	4432	female	40	Y

3. 排序(按索引，按数值)

Excel中可以通过数据目录下的排序按钮直接对数据表进行排序，比较简单。Python中需要使用ort_values函数和sort_index函数完成排序。



异步社区会员 lizhimiao(18926120246) 专享 尊重版权

在Python中，既可以按索引对数据表进行排序，也可以看制定列的数值进行排序。首先我们按age列中用户的年龄对数据表进行排序。使用的函数为sort_values。

```
#按特定列的值排序
df_inner.sort_values(by=['age'])
```

	id	date	city	category	age	price	gender	m-point	pay
0	1001	2013-01-02	beijing	100-A	23	1200	male	10	Y
3	1004	2013-01-05	shenzhen	110-C	32	5433	female	40	Y
5	1006	2013-01-07	beijing	130-F	32	4432	female	40	Y
4	1005	2013-01-06	shanghai	210-A	34	3299	male	40	N
1	1002	2013-01-03	shanghai	100-B	44	3299	female	12	N
2	1003	2013-01-04	guangzhou	110-A	54	2133	male	20	Y

Sort_index函数用来将数据表按索引列的值进行排序。

```
#按索引列排序
df_inner.sort_index()
```

	id	date	city	category	age	price	gender	m-point	pay
0	1001	2013-01-02	beijing	100-A	23	1200	male	10	Y
1	1002	2013-01-03	shanghai	100-B	44	3299	female	12	N
2	1003	2013-01-04	guangzhou	110-A	54	2133	male	20	Y
3	1004	2013-01-05	shenzhen	110-C	32	5433	female	40	Y
4	1005	2013-01-06	shanghai	210-A	34	3299	male	40	N
5	1006	2013-01-07	beijing	130-F	32	4432	female	40	Y

4. 数据分组

异步社区会员 lizhimiao(18926120246) 专享 尊重版权

欢迎加入非盈利Python编程学习交流QQ群783462347，群里免费提供500+本Python书籍！

Excel中可以通过VLOOKUP函数进行近似匹配来完成对数值的分组，或者使用“数据透视表”来完成分组。相应的 Python中使用where函数完成数据分组。

Where函数用来对数据进行判断和分组，下面的代码中我们对price列的值进行判断，将符合条件的分为一组，不符合条件的分为另一组，并使用group字段进行标记。

```
#如果price列的值>3000，group列显示high，否则显示low
df_inner['group'] = np.where(df_inner['price'] > 3000,'high','low
')
```

	id	date	city	category	age	price	gender	m-point	pay	group
0	1001	2013-01-02	beijing	100-A	23	1200	male	10	Y	low
3	1004	2013-01-05	shenzhen	110-C	32	5433	female	40	Y	high
5	1006	2013-01-07	beijing	130-F	32	4432	female	40	Y	high
4	1005	2013-01-06	shanghai	210-A	34	3299	male	40	N	high
1	1002	2013-01-03	shanghai	100-B	44	3299	female	12	N	high
2	1003	2013-01-04	guangzhou	110-A	54	2133	male	20	Y	low

除了where函数以外，还可以对多个字段的值进行判断后对数据进行分组，下面的代码中对city列等于beijing并且price列大于等于4000的数据标记为1。

```
#对复合多个条件的数据进行分组标记
df_inner.loc[(df_inner['city'] == 'beijing') & (df_inner['price']
 >= 4000), 'sign']=1
```


异步社区会员 lizhimiao(18926120246) 专享 尊重版权

	id	date	city	category	age	price	gender	m-point	pay	group	sign
0	1001	2013-01-02	beijing	100-A	23	1200	male	10	Y	low	NaN
3	1004	2013-01-05	shenzhen	110-C	32	5433	female	40	Y	high	NaN
5	1006	2013-01-07	beijing	130-F	32	4432	female	40	Y	high	1.0
4	1005	2013-01-06	shanghai	210-A	34	3299	male	40	N	high	NaN
1	1002	2013-01-03	shanghai	100-B	44	3299	female	12	N	high	NaN
2	1003	2013-01-04	guangzhou	110-A	54	2133	male	20	Y	low	NaN

5. 数据分列

与数据分组相反的是对数值进行分列，Excel中的数据目录下提供“分列”功能。在Python中使用split函数实现分列。



在数据表中category列中的数据包含有两个信息，前面的数字为类别id，后面的字母为size值。中间以连字符进行连接。我们使用split函数对这个字段进行拆分，并将拆分后的数据表匹配回原数据表中。

```
#对category字段的值依次进行分列，并创建数据表，索引值为df_inner的索引列，列名称为category和size
pd.DataFrame((x.split('-') for x in df_inner['category']),index=df_inner.index,columns=['category','size'])
```


异步社区会员 lizhimiao(18926120246) 专享 尊重版权

	category	size
0	100	A
3	110	C
5	130	F
4	210	A
1	100	B
2	110	A

```
#将完成分列后的数据表与原df_inner数据表进行匹配
df_inner=pd.merge(df_inner,split,right_index=True, left_index=True)
```

	id	date	city	category	age	price	gender	m-point	pay	group	sign	category_1	size
0	1001	2013-01-02	beijing	100-A	23	1200	male	10	Y	low	NaN	100	A
3	1004	2013-01-05	shenzhen	110-C	32	5433	female	40	Y	high	NaN	110	C
5	1006	2013-01-07	beijing	130-F	32	4432	female	40	Y	high	1.0	130	F
4	1005	2013-01-06	shanghai	210-A	34	3299	male	40	N	high	NaN	210	A
1	1002	2013-01-03	shanghai	100-B	44	3299	female	12	N	high	NaN	100	B
2	1003	2013-01-04	guangzhou	110-A	54	2133	male	20	Y	low	NaN	110	A

异步社区会员 lizhimiao(18926120246) 专享 尊重版权

第5章 数据提取

数据提取，也是数据分析中最常见的一个工作。这部分主要使用3个函数，即loc、iloc和ix。loc函数按标签值进行提取，iloc函数按位置进行提取，ix函数可以同时按标签和位置进行提取。下面介绍每一种函数的使用方法。

1. 按标签提取(loc)

Loc函数按数据表的索引标签进行提取，下面的代码中提取了索引列为3的单条数据。

```
#按索引提取单行的数值
df_inner.loc[3]
id                          1004
date          2013-01-05 00:00:00
city                     shenzhen
category                    110-C
age                            32
price                        5433
gender                     female
m-point                        40
pay                             Y
group                        high
sign                          NaN
category_1                    110
size                            C
Name: 3, dtype: object
```

使用冒号可以限定提取数据的范围，冒号前面为开始的标签值，

异步社区会员 lizhimiao(18926120246) 专享 尊重版权

后面为结束的标签值。下面提取了0到5的数据行。

```
#按索引提取区域行数值
df_inner.loc[0:5]
```

	id	date	city	category	age	price	gender	m-point	pay	group	sign	category_1	size
0	1001	2013-01-02	beijing	100-A	23	1200	male	10	Y	low	NaN	100	A
3	1004	2013-01-05	shenzhen	110-C	32	5433	female	40	Y	high	NaN	110	C
5	1006	2013-01-07	beijing	130-F	32	4432	female	40	Y	high	1.0	130	F

Reset_index函数用于恢复索引，这里我们重新将date字段的日期设置为数据表的索引，并按日期进行数据提取。

```
#重设索引
df_inner.reset_index()
```

	index	id	date	city	category	age	price	gender	m-point	pay	group	sign	category_1	size
0	0	1001	2013-01-02	beijing	100-A	23	1200	male	10	Y	low	NaN	100	A
1	3	1004	2013-01-05	shenzhen	110-C	32	5433	female	40	Y	high	NaN	110	C
2	5	1006	2013-01-07	beijing	130-F	32	4432	female	40	Y	high	1.0	130	F
3	4	1005	2013-01-06	shanghai	210-A	34	3299	male	40	N	high	NaN	210	A
4	1	1002	2013-01-03	shanghai	100-B	44	3299	female	12	N	high	NaN	100	B
5	2	1003	2013-01-04	guangzhou	110-A	54	2133	male	20	Y	low	NaN	110	A

```
#设置日期为索引
df_inner=df_inner.set_index('date')
```

	id	city	category	age	price	gender	m-point	pay	group	sign	category_1	size
date												
2013-01-02	1001	beijing	100-A	23	1200	male	10	Y	low	NaN	100	A
2013-01-05	1004	shenzhen	110-C	32	5433	female	40	Y	high	NaN	110	C
2013-01-07	1006	beijing	130-F	32	4432	female	40	Y	high	1.0	130	F
2013-01-06	1005	shanghai	210-A	34	3299	male	40	N	high	NaN	210	A
2013-01-03	1002	shanghai	100-B	44	3299	female	12	N	high	NaN	100	B
2013-01-04	1003	guangzhou	110-A	54	2133	male	20	Y	low	NaN	110	A

异步社区会员 lizhimiao(18926120246) 专享 尊重版权

使用冒号限定提取数据的范围，冒号前面为空表示从0开始。提取所有2013年1月4日以前的数据。

```
#提取4日之前的所有数据
df_inner[:'2013-01-04']
```

	id	city	category	age	price	gender	m-point	pay	group	sign	category_1	size
date												
2013-01-02	1001	beijing	100-A	23	1200	male	10	Y	low	NaN	100	A
2013-01-03	1002	shanghai	100-B	44	3299	female	12	N	high	NaN	100	B
2013-01-04	1003	guangzhou	110-A	54	2133	male	20	Y	low	NaN	110	A

2. 按位置提取(iloc)

使用iloc函数按位置对数据表中的数据进行提取，这里冒号前后的数字不再是索引的标签名称，而是数据所在的位置，从0开始。

```
使用iloc按位置区域提取数据
df_inner.iloc[:3,:2]
```

	id	city
date		
2013-01-02	1001	beijing
2013-01-05	1004	shenzhen
2013-01-07	1006	beijing

iloc函数除了可以按区域提取数据，还可以按位置逐条提取，前面方括号中的0,2,5表示数据所在行的位置，后面方括号中的数表示所在列的位置。

```
#使用iloc按位置单独提取数据
df_inner.iloc[[0,2,5],[4,5]]
```


异步社区会员 lizhimiao(18926120246) 专享 尊重版权

	price	gender
date		
2013-01-02	1200	male
2013-01-07	4432	female
2013-01-04	2133	male

3. 按标签和位置提取（ix）

ix是loc和iloc的混合，既能按索引标签提取，也能按位置进行数据提取。下面代码中行的位置按索引日期设置，列按位置设置。

```
#使用ix按索引标签和位置混合提取数据
df_inner.ix[:'2013-01-03',:4]
```

	id	city	category	age
date				
2013-01-02	1001	beijing	100-A	23
2013-01-03	1002	shanghai	100-B	44

4. 按条件提取（区域和条件值）

除了按标签和位置提起数据以外，还可以按具体的条件进行数据。下面使用loc和isin两个函数配合使用，按指定条件对数据进行提取。

使用isin函数对city中的值是否为beijing进行判断。

```
#判断city列的值是否为beijing
df_inner['city'].isin(['beijing'])

date
2013-01-02     True
```


异步社区会员 lizhimiao(18926120246) 专享 尊重版权

```
2013-01-05    False
2013-01-07     True
2013-01-06    False
2013-01-03    False
2013-01-04    False
Name: city, dtype: bool
```

将isin函数嵌套到loc的数据提取函数中，将判断结果为Ture数据提取出来。这里我们把判断条件改为city值是否为beijing和shanghai。如果是就把这条数据提取出来。

```
#先判断city列里是否包含beijing和shanghai，然后将复合条件的数据提取出来。
df_inner.loc[df_inner['city'].isin(['beijing','shanghai'])]
```

	id	city	category	age	price	gender	m-point	pay	group	sign	category_1	size
date												
2013-01-02	1001	beijing	100-A	23	1200	male	10	Y	low	NaN	100	A
2013-01-07	1006	beijing	130-F	32	4432	female	40	Y	high	1.0	130	F
2013-01-06	1005	shanghai	210-A	34	3299	male	40	N	high	NaN	210	A
2013-01-03	1002	shanghai	100-B	44	3299	female	12	N	high	NaN	100	B

数值提取还可以完成类似数据分列的工作，从合并的数值中提取出制定的数值。

```
category=df_inner['category']
0    100-A
3    110-C
5    130-F
4    210-A
1    100-B
2    110-A
Name: category, dtype: object


#提取前三个字符，并生成数据表
pd.DataFrame(category.str[:3])
```


异步社区会员 lizhimiao(18926120246) 专享 尊重版权

	category
0	100
3	110
5	130
4	210
1	100
2	110

异步社区会员 lizhimiao(18926120246) 专享 尊重版权

第6章 数据筛选

使用与，或，非三个条件配合大于，小于和等于对数据进行筛选，并进行计数和求和。与Excel中的筛选功能和countifs和sumifs功能相似。

按条件筛选（与、或、非）

Excel数据目录下提供了“筛选”功能，用于对数据表按不同的条件进行筛选。Python中使用loc函数配合筛选条件来完成筛选功能。配合sum和count函数还能实现Excel中sumif和countif函数的功能。



使用“与”条件进行筛选，条件是年龄大于25岁，并且城市为beijing。筛选后只有一条数据符合要求。

```
#使用“与”条件进行筛选
df_inner.loc[(df_inner['age'] > 25) & (df_inner['city'] == 'beiji
ng'), ['id','city','age','category','gender']]/
```


异步社区会员 lizhimiao(18926120246) 专享 尊重版权

	id	city	age	category	gender
date					
2013-01-07	1006	beijing	32	130-F	female

使用“或”条件进行筛选，年龄大于25岁或城市为beijing。筛选后有6条数据符合要求。

```
#使用“或”条件筛选
df_inner.loc[(df_inner['age'] > 25) | (df_inner['city'] == 'beiji
ng'), ['id','city','age','category','gender']].sort(['age'])
```

	id	city	age	category	gender
date					
2013-01-02	1001	beijing	23	100-A	male
2013-01-05	1004	shenzhen	32	110-C	female
2013-01-07	1006	beijing	32	130-F	female
2013-01-06	1005	shanghai	34	210-A	male
2013-01-03	1002	shanghai	44	100-B	female
2013-01-04	1003	guangzhou	54	110-A	male

在前面的代码后增加price字段以及sum函数，按筛选后的结果将price字段值进行求和，相当于Excel中sumifs的功能。

```
#对筛选后的数据按price字段进行求和
df_inner.loc[(df_inner['age'] > 25) | (df_inner['city'] == 'beiji
ng'), ['id','city','age','category','gender','price']].sort(['age
']).price.sum()

19796
```

使用“非”条件进行筛选，城市不等于beijing。符合条件的数据有

异步社区会员 lizhimiao(18926120246) 专享 尊重版权

4条。将筛选结果按id列进行排序。

```
#使用“非”条件进行筛选
df_inner.loc[(df_inner['city'] != 'beijing'), ['id','city','age',
'category','gender']].sort(['id'])
```

	id	city	age	category	gender
date					
2013-01-03	1002	shanghai	44	100-B	female
2013-01-04	1003	guangzhou	54	110-A	male
2013-01-05	1004	shenzhen	32	110-C	female
2013-01-06	1005	shanghai	34	210-A	male

在前面的代码后面增加city列，并使用count函数进行计数。相当于Excel中的countifs函数的功能。

```
#对筛选后的数据按city列进行计数
df_inner.loc[(df_inner['city'] != 'beijing'), ['id','city','age',
'category','gender']].sort(['id']).city.count()
4
```

还有一种筛选的方式是用query函数。下面是具体的代码和筛选结果。

```
#使用query函数进行筛选
df_inner.query('city == ["beijing", "shanghai"]')
```

	id	city	category	age	price	gender	m-point	pay	group	sign	category_1	size
date												
2013-01-02	1001	beijing	100-A	23	1200	male	10	Y	low	NaN	100	A
2013-01-07	1006	beijing	130-F	32	4432	female	40	Y	high	1.0	130	F
2013-01-06	1005	shanghai	210-A	34	3299	male	40	N	high	NaN	210	A
2013-01-03	1002	shanghai	100-B	44	3299	female	12	N	high	NaN	100	B

异步社区会员 lizhimiao(18926120246) 专享 尊重版权

在前面的代码后增加price字段和sum函数。对筛选后的price字段进行求和，相当于Excel中的sumifs函数的功能。

```
#对筛选后的结果按price进行求和
df_inner.query('city == ["beijing", "shanghai"]').price.sum()
12230
```


异步社区会员 lizhimiao(18926120246) 专享 尊重版权

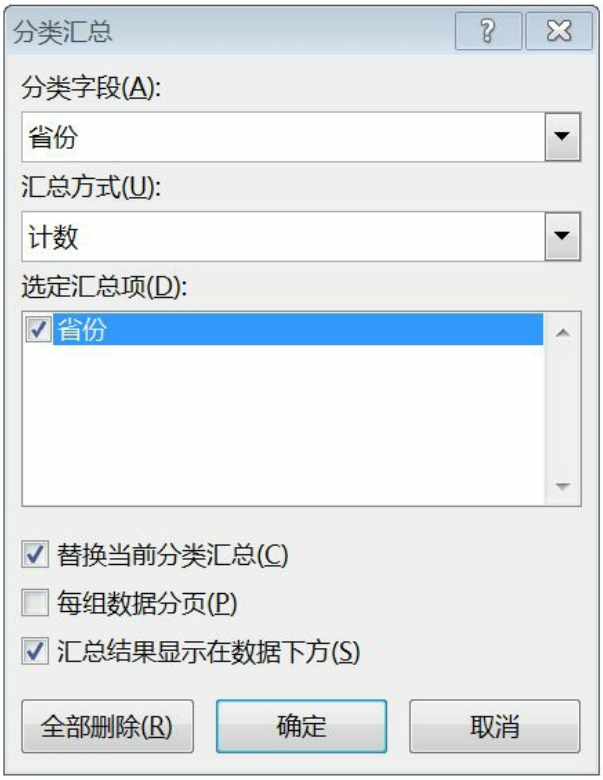
第7章 数据汇总

本章主要讲解如何对数据进行分类汇总。Excel中使用分类汇总和数据透视可以按特定维度对数据进行汇总，Python中使用的主要函数是groupby和pivot_table。下面分别介绍这两个函数的使用方法。

1. 分类汇总

Excel的数据目录下提供了“分类汇总”功能，可以按指定的字段和汇总方式对数据表进行汇总。Python中通过Groupby函数完成相应的操作，并可以支持多级分类汇总。

异步社区会员 lizhimiao(18926120246) 专享 尊重版权



Groupby是进行分类汇总的函数，使用方法很简单，制定要分组的列名称就可以，也可以同时制定多个列名称，groupby按列名称出现的顺序进行分组。同时要制定分组后的汇总方式，常见的是计数和求和两种。

```
#对所有列进行计数汇总
df_inner.groupby('city').count()/
```

	id	category	age	price	gender	m-point	pay	group	sign	category_1	size
city											
beijing	2	2	2	2	2	2	2	2	1	2	2
guangzhou	1	1	1	1	1	1	1	1	0	1	1
shanghai	2	2	2	2	2	2	2	2	0	2	2
shenzhen	1	1	1	1	1	1	1	1	0	1	1

可以在groupby中设置列名称来对特定的列进行汇总。下面的代

异步社区会员 lizhimiao(18926120246) 专享 尊重版权

码中按城市对id字段进行汇总计数。

```
#对特定的ID列进行计数汇总
df_inner.groupby('city')['id'].count()
city
beijing      2
guangzhou    1
shanghai     2
shenzhen     1
Name: id, dtype: int64
```

在前面的基础上增加第二个列名称，分布对city和size两个字段进行计数汇总。

```
#对两个字段进行汇总计数
df_inner.groupby(['city','size'])['id'].count()
city       size
beijing    A       1
           F       1
guangzhou  A       1
shanghai   A       1
           B       1
shenzhen   C       1
Name: id, dtype: int64
```

除了计数和求和外，还可以对汇总后的数据同时按多个维度进行计算，下面的代码中按城市对price字段进行汇总，并分别计算price的数量，总金额和平均金额。

```
#对city字段进行汇总并计算price的合计和均值。
df_inner.groupby('city')['price'].agg([len,np.sum, np.mean])
```


异步社区会员 lizhimiao(18926120246) 专享 尊重版权

	len	sum	mean
city			
beijing	2	5632	2816
guangzhou	1	2133	2133
shanghai	2	6598	3299
shenzhen	1	5433	5433

2. 数据透视

Excel中的插入目录下提供“数据透视表”功能对数据表按特定维度进行汇总。Python中也提供了数据透视表功能。通过pivot_table函数实现同样的效果。



数据透视表也是常用的一种数据分类汇总方式，并且功能上比groupby要强大一些。下面的代码中设定city为行字段，size为列字段，price为值字段。分别计算price的数量和金额并且按行与列进行汇总。

```
#数据透视表
pd.pivot_table(df_inner,index=["city"],values=["price"],columns=["size"],aggfunc=[len,np.sum],fill_value=0,margins=True)
```


异步社区会员 lizhimiao(18926120246) 专享 尊重版权

	len					sum				
	price					price				
size	A	B	C	F	All	A	B	C	F	All
city										
beijing	1.0	0.0	0.0	1.0	2.0	1200.0	0.0	0.0	4432.0	5632.0
guangzhou	1.0	0.0	0.0	0.0	1.0	2133.0	0.0	0.0	0.0	2133.0
shanghai	1.0	1.0	0.0	0.0	2.0	3299.0	3299.0	0.0	0.0	6598.0
shenzhen	0.0	0.0	1.0	0.0	1.0	0.0	0.0	5433.0	0.0	5433.0
All	3.0	1.0	1.0	1.0	6.0	6632.0	3299.0	5433.0	4432.0	19796.0

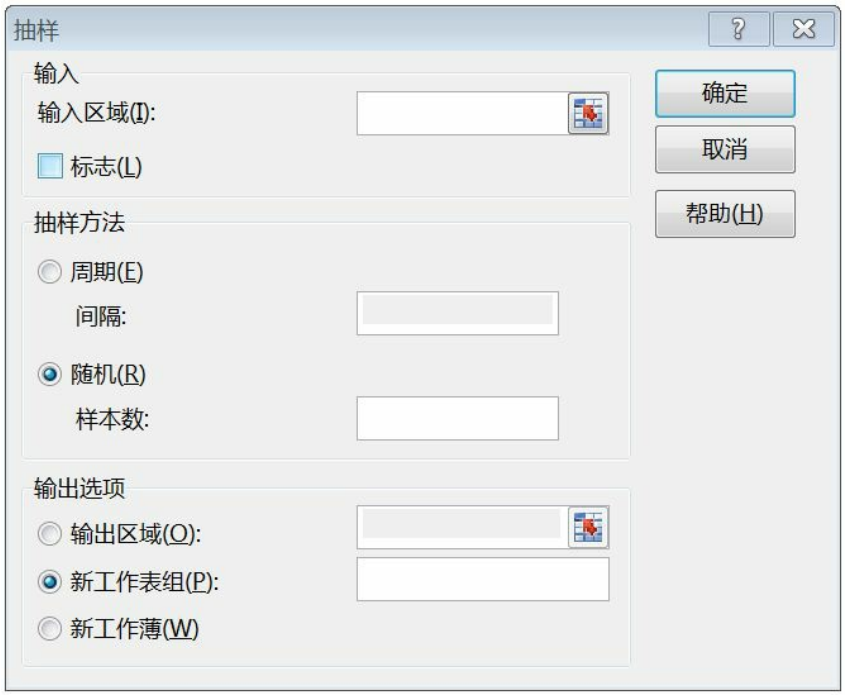
异步社区会员 lizhimiao(18926120246) 专享 尊重版权

第8章 数据统计

本章主要介绍数据采样，标准差，协方差和相关系数的使用方法。

1. 数据采样

Excel的数据分析功能中提供了数据抽样的功能，如下图所示。Python通过sample函数完成数据采样。



Sample是进行数据采样的函数，设置n的数量就可以了。函数自动返回参与的结果。

异步社区会员 lizhimiao(18926120246) 专享 尊重版权

```
#简单的数据采样
df_inner.sample(n=3)
```

	id	city	category	age	price	gender	m-point	pay	group	sign	category_1	size
date												
2013-01-07	1006	beijing	130-F	32	4432	female	40	Y	high	1.0	130	F
2013-01-02	1001	beijing	100-A	23	1200	male	10	Y	low	NaN	100	A
2013-01-04	1003	guangzhou	110-A	54	2133	male	20	Y	low	NaN	110	A

Weights参数是采样的权重，通过设置不同的权重可以更改采样的结果，权重高的数据将更有希望被选中。这里手动设置6条数据的权重值。将前面4个设置为0，后面两个分别设置为0.5。

```
#手动设置采样权重
weights = [0, 0, 0, 0, 0.5, 0.5]
df_inner.sample(n=2, weights=weights)
```

	id	city	category	age	price	gender	m-point	pay	group	sign	category_1	size
date												
2013-01-02	1001	beijing	100-A	23	1200	male	10	Y	low	NaN	100	A
2013-01-05	1004	shenzhen	110-C	32	5433	female	40	Y	high	NaN	110	C
2013-01-07	1006	beijing	130-F	32	4432	female	40	Y	high	1.0	130	F
2013-01-06	1005	shanghai	210-A	34	3299	male	40	N	high	NaN	210	A
2013-01-03	1002	shanghai	100-B	44	3299	female	12	N	high	NaN	100	B
2013-01-04	1003	guangzhou	110-A	54	2133	male	20	Y	low	NaN	110	A

从采样结果中可以看出，后两条权重高的数据被选中。

	id	city	category	age	price	gender	m-point	pay	group	sign	category_1	size
date												
2013-01-04	1003	guangzhou	110-A	54	2133	male	20	Y	low	NaN	110	A
2013-01-03	1002	shanghai	100-B	44	3299	female	12	N	high	NaN	100	B

Sample函数中还有一个参数replace，用来设置采样后是否放回。

异步社区会员 lizhimiao(18926120246) 专享 尊重版权

```
#采样后不放回
df_inner.sample(n=6, replace=False)
```

	id	city	category	age	price	gender	m-point	pay	group	sign	category_1	size
date												
2013-01-06	1005	shanghai	210-A	34	3299	male	40	N	high	NaN	210	A
2013-01-03	1002	shanghai	100-B	44	3299	female	12	N	high	NaN	100	B
2013-01-04	1003	guangzhou	110-A	54	2133	male	20	Y	low	NaN	110	A
2013-01-02	1001	beijing	100-A	23	1200	male	10	Y	low	NaN	100	A
2013-01-07	1006	beijing	130-F	32	4432	female	40	Y	high	1.0	130	F
2013-01-05	1004	shenzhen	110-C	32	5433	female	40	Y	high	NaN	110	C

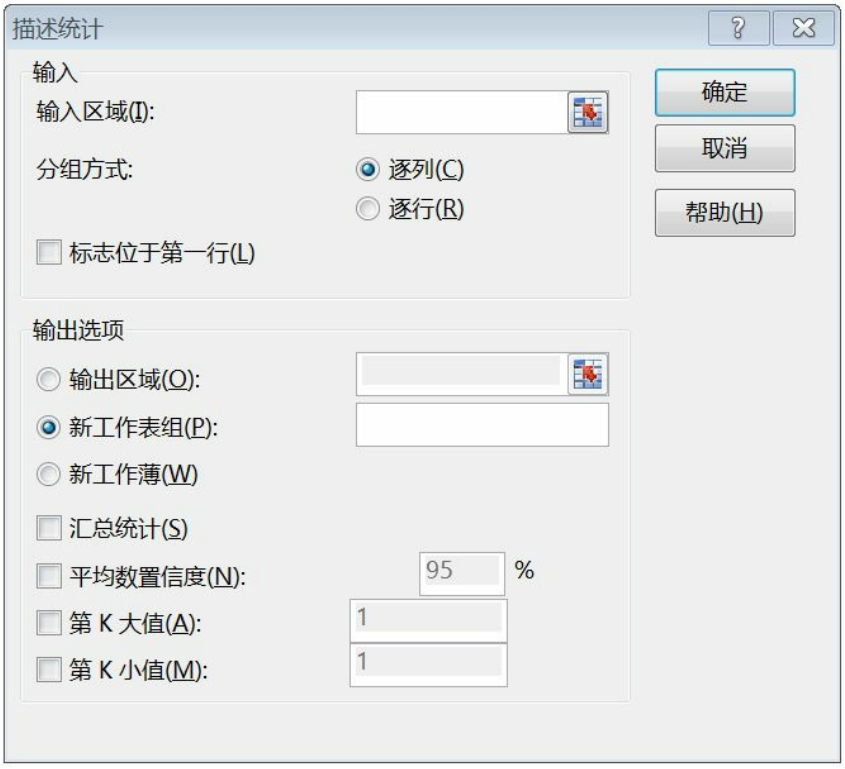
```
#采样后放回
df_inner.sample(n=6, replace=True)
```

	id	city	category	age	price	gender	m-point	pay	group	sign	category_1	size
date												
2013-01-05	1004	shenzhen	110-C	32	5433	female	40	Y	high	NaN	110	C
2013-01-02	1001	beijing	100-A	23	1200	male	10	Y	low	NaN	100	A
2013-01-03	1002	shanghai	100-B	44	3299	female	12	N	high	NaN	100	B
2013-01-07	1006	beijing	130-F	32	4432	female	40	Y	high	1.0	130	F
2013-01-05	1004	shenzhen	110-C	32	5433	female	40	Y	high	NaN	110	C
2013-01-06	1005	shanghai	210-A	34	3299	male	40	N	high	NaN	210	A

2. 描述统计

Excel中的数据分析中提供了描述统计的功能。Python中可以通过Describe对数据进行描述统计。

异步社区会员 lizhimiao(18926120246) 专享 尊重版权



Describe函数是进行描述统计的函数，自动生成数据的数量，均值，标准差等数据。下面的代码中对数据表进行描述统计，并使用round函数设置结果显示的小数位。并对结果数据进行转置。

```
#数据表描述性统计
df_inner.describe().round(2).T
```

	count	mean	std	min	25%	50%	75%	max
id	6.0	1003.50	1.87	1001.0	1002.25	1003.5	1004.75	1006.0
age	6.0	36.50	10.88	23.0	32.00	33.0	41.50	54.0
price	6.0	3299.33	1523.35	1200.0	2424.50	3299.0	4148.75	5433.0
m-point	6.0	27.00	14.63	10.0	14.00	30.0	40.00	40.0
sign	1.0	1.00	NaN	1.0	1.00	1.0	1.00	1.0

3. 标准差

异步社区会员 lizhimiao(18926120246) 专享 尊重版权

Python中的Std函数用来接算特定数据列的标准差。

```
#标准差
df_inner['price'].std()
1523.3516556155596
```

4. 协方差

Excel中的数据分析功能中提供协方差的计算，Python中通过cov函数计算两个字段或数据表中各字段间的协方差。



Cov函数用来计算两个字段间的协方差，可以只对特定字段进行计算，也可以对整个数据表中各个列之间进行计算。

```
#两个字段间的协方差
df_inner['price'].cov(df_inner['m-point'])
17263.200000000001
```

```
#数据表中所有字段间的协方差
df_inner.cov()
```


异步社区会员 lizhimiao(18926120246) 专享 尊重版权

	id	age	price	m-point	sign
id	3.5	-0.7	1.946000e+03	25.4	NaN
age	-0.7	118.3	-1.354000e+03	-31.0	NaN
price	1946.0	-1354.0	2.320600e+06	17263.2	NaN
m-point	25.4	-31.0	1.726320e+04	214.0	NaN
sign	NaN	NaN	NaN	NaN	NaN

5. 相关分析

Excel的数据分析功能中提供了相关系数的计算功能，Python中则通过corr函数完成相关分析的操作，并返回相关系数。



Corr函数用来计算数据间的相关系数，可以单独对特定数据进行计算，也可以对整个数据表中各个列进行计算。相关系数在-1到1之间，接近1为正相关，接近-1为负相关，0为不相关。

```
#相关性分析
df_inner['price'].corr(df_inner['m-point'])
0.77466555617085264
```


异步社区会员 lizhimiao(18926120246) 专享 尊重版权

```
#数据表相关性分析
df_inner.corr()
```

	id	age	price	m-point	sign
id	1.000000	-0.034401	0.682824	0.928096	NaN
age	-0.034401	1.000000	-0.081720	-0.194833	NaN
price	0.682824	-0.081720	1.000000	0.774666	NaN
m-point	0.928096	-0.194833	0.774666	1.000000	NaN
sign	NaN	NaN	NaN	NaN	NaN

异步社区会员 lizhimiao(18926120246) 专享 尊重版权

第9章 数据输出

处理和分析完的数据可以输出为xlsx格式和csv格式。

1. 写入Excel

```
#输出到Excel格式
df_inner.to_Excel('Excel_to_Python.xlsx', sheet_name='bluewhale_cc')
```

	A	B	C	D	E	F	G	H	I	J	K	L	M
1	date	id	city	category	age	price	gender	m-point	pay	group	sign	category_1	size
2	2013-01-02 00:00:00	1001	beijing	100-A	23	1200	male	10	Y	low		100	A
3	2013-01-05 00:00:00	1004	shenzhen	110-C	32	5433	female	40	Y	high		110	C
4	2013-01-07 00:00:00	1006	beijing	130-F	32	4432	female	40	Y	high	1	130	F
5	2013-01-06 00:00:00	1005	shanghai	210-A	34	3299	male	40	N	high		210	A
6	2013-01-03 00:00:00	1002	shanghai	100-B	44	3299	female	12	N	high		100	B
7	2013-01-04 00:00:00	1003	guangzhou	110-A	54	2133	male	20	Y	low		110	A

2. 写入csv

```
#输出到CSV格式
df_inner.to_csv('Excel_to_Python.csv')
```


异步社区会员 lizhimiao(18926120246) 专享 尊重版权

案例　990万次骑行：纽约自行车共享系统分析



Citi Bike是纽约市在2013年5月27日启动的一项自行车共享计划，由“花旗银行”(Citi Bank)赞助并取名为“花旗单车”(Citi Bike)。在曼哈顿，布鲁克林，皇后区和泽西市有8,000辆自行车和500个车站。为纽约的居民和游客提供一种方便快捷，并且省钱的出行方式。



人们随处都能借到Citi Bank，并在他们的目的地归还。使用Citi Bank的方法很简单，购买会员资格，然后在附近查找可以使用的Citi Bank，使用会员key解锁，在有效时间内(30或45分钟)归还并锁定Citi

异步社区会员 lizhimiao(18926120246) 专享 尊重版权

Bank。由于每个Citi Bank的租赁站点都有GPS位置信息，因此可以记录到用户租赁和骑行过程中的数据。“Citi Bank”官网提供了用户每一次骑行的数据，包括租赁开始及结束的位置及时间，整个骑行过程的时间，自行车ID，以及用户的性别和出生日期等数据。我们将使用python对“Citi Bank”2015年的数据进行分析，以了解纽约自行车共享计划的情况。并回答诸如谁在使用Citi Bike？他们什么时间开始骑行？每次骑行的时间和距离，以及哪些骑行线路最受欢迎等问题。

开始前的准备工作

首先我们将需要使用的库文件导入到python中，这里包括numpy、pandas、datetime和用于数据可视化的pyplot，后面根据不同的分析方向我们还会陆续导入其他的库文件。

```
import numpy as np
import pandas as pd
import time,datetime
import matplotlib.pyplot as plt

#读取Citi Bike的数据并创建数据表
cb1=pd.DataFrame(pd.read_csv('201501-citibike-tripdata.csv'))
```

Citi Bike的数据是按月存储的，因此我们需要逐月读取并将各个月份的数据进行拼接。

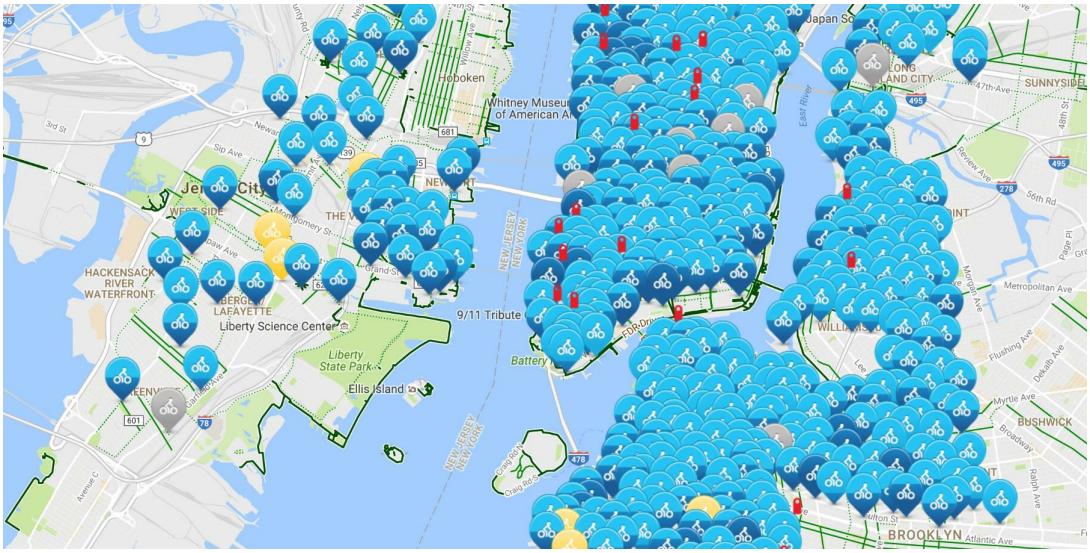
```
#对导入的数据表进行拼接，汇总后的数据表名为cb
cb=cb1.append(cb2,ignore_index=False)

#查看数据表维度
cb.shape
(9937969, 15)
```


异步社区会员 lizhimiao(18926120246) 专享 尊重版权

2015年Citi Bike数据共包含993万行，15列数据。其中每一行数据都代表一次“Citi Bank”的租借和骑行记录。

惊人的990万次骑行



我们首先对2015年“Citi Bank”的数据中的几个关键指标进行统计，在一年中共有497个租赁点，也就是车站，8477辆自行车被使用。自行车的使用次数更是高达993万次，平均算下来每辆自行车每天被租借3.21次。可见人们对“Citi Bank”的热情。而每次的骑行时间平均为16.13分钟。

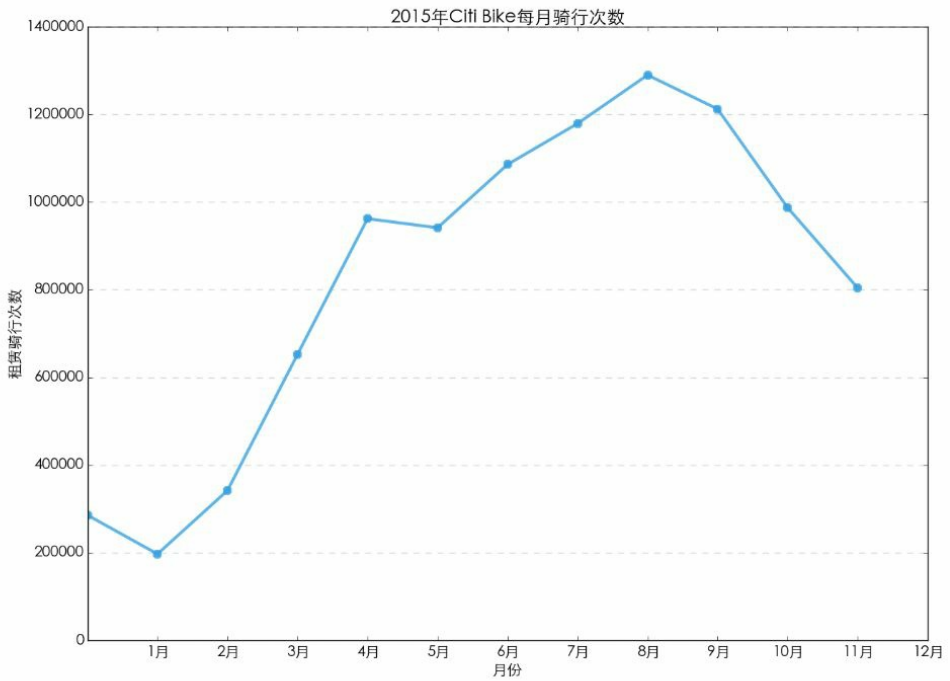
租赁点数量	自行车数量	租赁次数	租赁频率	租赁时长
497个	8,477辆	9,937,969次	3.21次/天	16.13分钟

以下为每个关键指标的统计代码和结果。

```
#唯一租赁点数量计数
len(cb['start station name'].unique())
497
```


异步社区会员 lizhimiao(18926120246) 专享 尊重版权

```
#唯一自行车ID计数
len(cb['bikeid'].unique())
8477
#骑行次数计数
cb['starttime'].count()
9937969
#每辆自行车租借频率
cb['bikeid'].count()/len(cb['bikeid'].unique())
1172.3450513153239
#每辆自行车每日租借频率
cb['bikeid'].count()/len(cb['bikeid'].unique())/365
3.2119042501789696
#每次租借平均时长(分钟)
cb['tripduration'].sum()/cb['bikeid'].count()/60
16.134794237132358
```



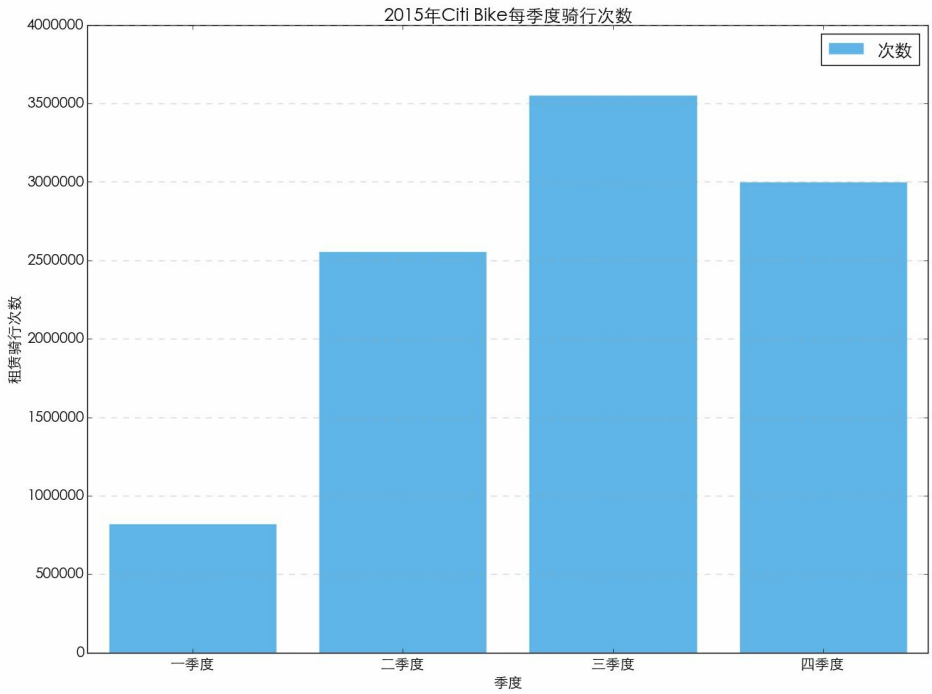
2015年的Citi Bike的使用量整体趋势由低到高，2月Citi Bike的使用量最低，然后使用量逐月增长，直到9月出现使用量最高值。11月使用量开始下降。这可能是由于季节和气温因素导致的。我们下面按季度对骑行数据进行汇总并进行对比。

异步社区会员 lizhimiao(18926120246) 专享 尊重版权

以下是按月汇总骑行数据并汇总折线图的代码。

```
#对starttime设置日期格式
cb['starttime']=pd.to_datetime(cb['starttime'])
#将starttime设置为数据表索引
cb = cb.set_index('starttime')
#按月对骑行数据进行计数
cb_month=cb.resample('M',how=len)
#提取汇总后的bikeid字段
group_cb_month=cb_month['bikeid']
#汇总按月汇总的骑行次数折线图
plt.rc('font', family='STXihei', size=15)
a=np.array([1,2,3,4,5,6,7,8,9,10,11,12])
plt.plot(group_cb_month,'g8',group_cb_month,'g-',color='#39A2E1',
linewidth=3,markeredgewidth=3,markeredgecolor='#39A2E1',alpha=0.8
)
plt.xlabel('月份')
plt.ylabel('租赁骑行次数')
plt.title('2015年Citi Bike每月骑行次数')
plt.grid( color='#95a5a6',linestyle='--', linewidth=1 ,axis='y',a
lpha=0.4)
plt.xticks(a, ('1月','2月','3月','4月','5月','6月','7月','8月','9月'
,'10月','11月','12月') )
plt.show()
```


异步社区会员 lizhimiao(18926120246) 专享 尊重版权



从2015年四个季度的数据来看，用户对Citi Bike的使用受季节因素的影响，第一季度气温最低，Citi Bike的使用量也较低。第二季度第三季度为夏秋交替，使用量最高。

```
#按季度对骑行数据进行计数
cb_quarterly=cb.resample('Q',how=len)
#提取按季度汇总后的bikeid字段
group_cb_quarterly=cb_quarterly['bikeid']
#绘制按季度汇总的骑行次数柱状图
plt.rc('font', family='STXihei', size=15)
a=np.array([1,2,3,4])
plt.bar([1,2,3,4],group_cb_quarterly,color='#39A2E1',alpha=0.8,al
ign='center',edgecolor='white')
plt.xlabel('季度')
plt.ylabel('租赁骑行次数')
plt.title('2015年Citi Bike每季度骑行次数')
plt.legend(['次数'], loc='upper right')
plt.grid(color='#95a5a6',linestyle='--', linewidth=1,axis='y',alp
ha=0.4)
plt.xticks(a,('一季度','二季度','三季度','四季度'))
```


异步社区会员 lizhimiao(18926120246) 专享 尊重版权

```
plt.show()
```

哪些人在使用Citi Bike？

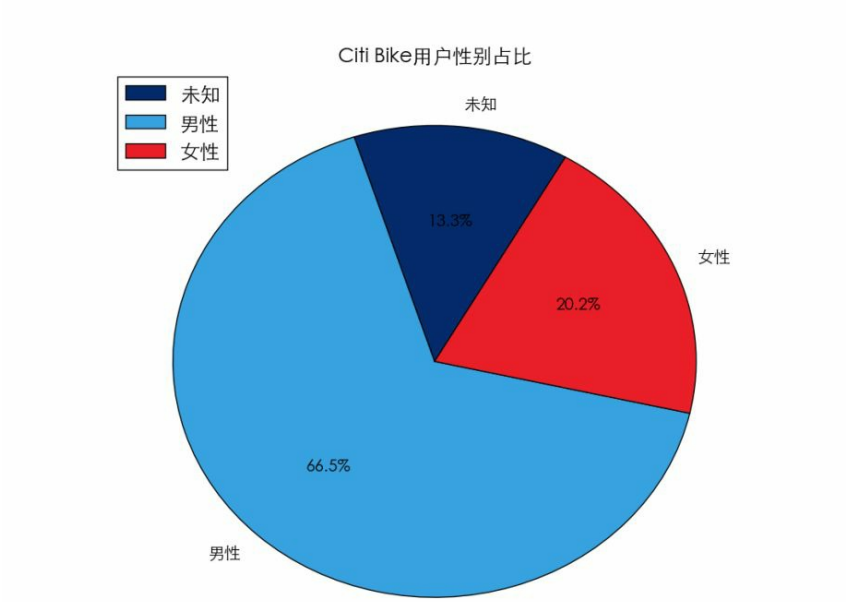


我们很好奇993万次骑行这个庞大的数据后面是哪些人在使用Citi Bike。由于获得的数据中只有用户性别，出生日期和会员类别的数据，因此我们仅从这三个维度对Citi Bike的用户进行简单的描述。

66%的男性用户使用Citi Bike

2015年使用Citi Bike服务的用户中，66%为男性用户，女性用户的占比为20%，另外还有13%的用户性别未知。这让我想起了北京自重25公斤的共享单车。

异步社区会员 lizhimiao(18926120246) 专享 尊重版权



以下是计算用户性别占比和绘制饼图的代码。

```
#按用户性别进行汇总并计算不同性别的占比
user_gender=cb.groupby('gender')['bikeid'].agg(len)/cb["bikeid"].
count()*100

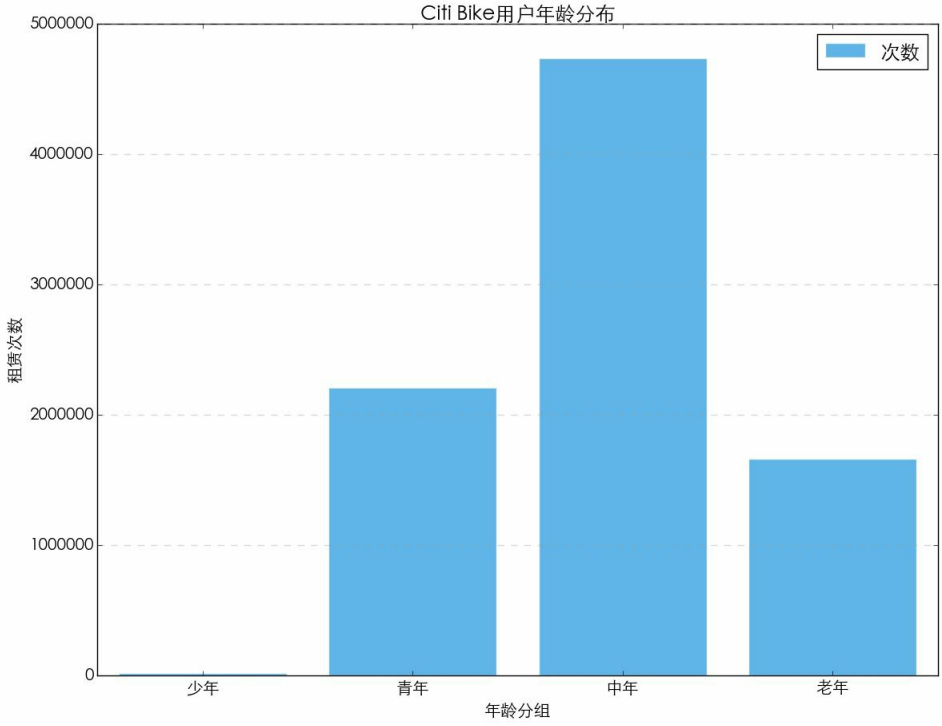
#汇总用户性别占比饼图
plt.rc('font', family='STXihei', size=15)
colors = ["#052B6C","#39A2E1","#EA1F29"]
name=['未知', '男性', '女性']
plt.pie(user_gender,labels=name,colors=colors,explode=(0, 0, 0),s
tartangle=60,autopct='%1.1f%%')
plt.title('Citi Bike用户性别占比')
plt.legend(['未知', '男性', '女性'], loc='upper left')
plt.show()
```

30～50岁用户热衷于骑行

我们将Citi Bike的用户年龄分为四组，0～18岁为少年组，18～30岁为青年组，30～50岁为中年组，50岁以上为老年组。其中少年组的

异步社区会员 lizhimiao(18926120246) 专享 尊重版权

租借和骑行次数最少，中年组的租借和骑行次数最高，其次为青年组。50岁以上的老年组也有相当数量的记录。



以下是对用户年龄分组和汇总柱状图的代码，用户年龄由出生日期和当前年份计算得出，其中包含部分极端值。

```
#查看出生日期的范围
cb['birth year'].min(),cb['birth year'].max()
(1885.0, 1999.0)
```

这里1885年很怪异，按照这个出生日期，2016年时用户的年龄已经达到了131岁。我们不知道其中的具体原因，但由于Citi Bike整体的数据较为规范不需要清洗，因此我们选择保留这个值。

```
#使用2015年与用户出生日期计算年龄
```


异步社区会员 lizhimiao(18926120246) 专享 尊重版权

```
cb['age']=2015-cb['birth year']
#用户最小年龄17岁，最大年龄131岁
cb['age'].min(),cb['age'].max()
(17.0, 131.0)

#对用户年龄进行分组
bins = [0, 18, 30, 50, 131]
group_age = ['少年', '青年', '中年', '老年']
cb['group_age'] = pd.cut(cb['age'], bins, labels=group_age)

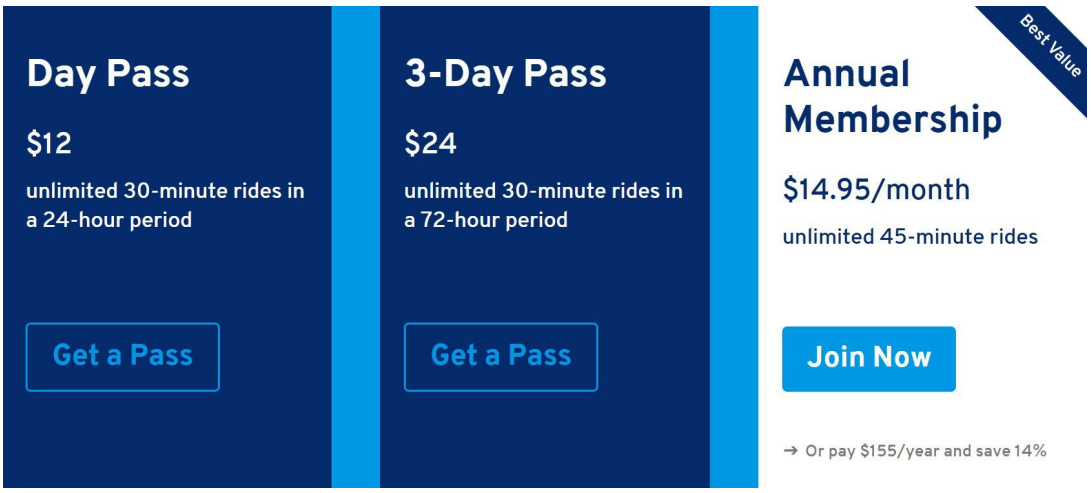
#按年龄分组对数据进行汇总
user_age=cb.groupby('group_age')['group_age'].agg(len)


#生成用户年龄分布柱状图
plt.rc('font', family='STXihei', size=15)
a=np.array([1,2,3,4])
plt.bar([1,2,3,4],user_age,color='#052B6C',alpha=0.8,align='cente
r',edgecolor='white')
plt.xlabel('年龄分组')
plt.ylabel('租赁次数')
plt.title('Citi Bike用户年龄分布')
plt.legend(['次数'], loc='upper right')
plt.grid(color='#95a5a6',linestyle='--', linewidth=1,axis='y',alp
ha=0.4)
plt.xticks(a,('少年','青年','中年','老年'))
plt.show()
```

86%的用户为年费会员

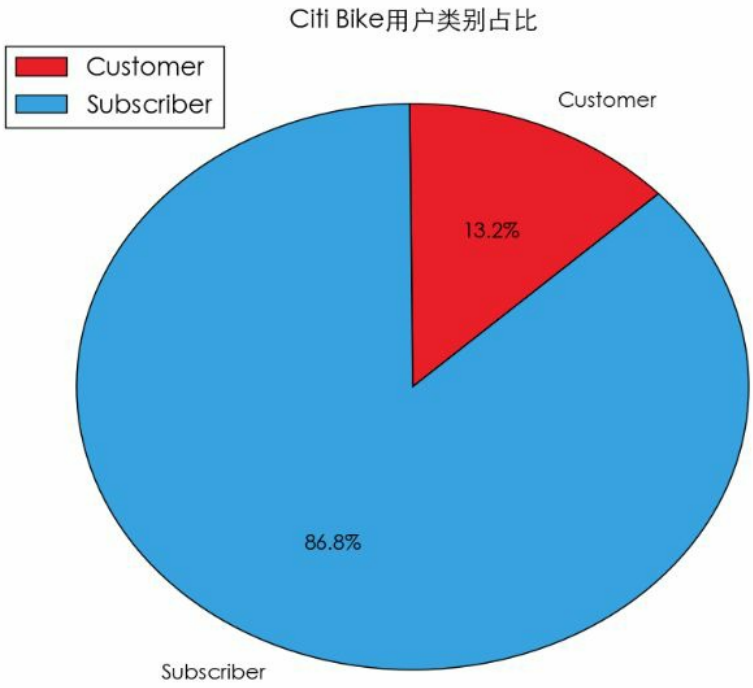
Ctii Bike对纽约本地居民，短期停留和游客提供了三种会员期限选择，按Citi Bike官网的推荐，年度会员适合于本地居民，也是最划算的一种租赁方式。3天的会员适合于短期停留或居住的用户，1天的会员适合于游客。因此我们也可以按不同的会员期限反向推断用户的身份。

异步社区会员 lizhimiao(18926120246) 专享 尊重版权



在2015年的数据表中，1天和3天的会员统称为Customer，年度会有成为Subscriber。从下面的占比数据中可以看出，绝大部分Citi Bike的用户为年度会员，占比高达86%。反向推测绝大部分用户为纽约本地常住居民。

异步社区会员 lizhimiao(18926120246) 专享 尊重版权



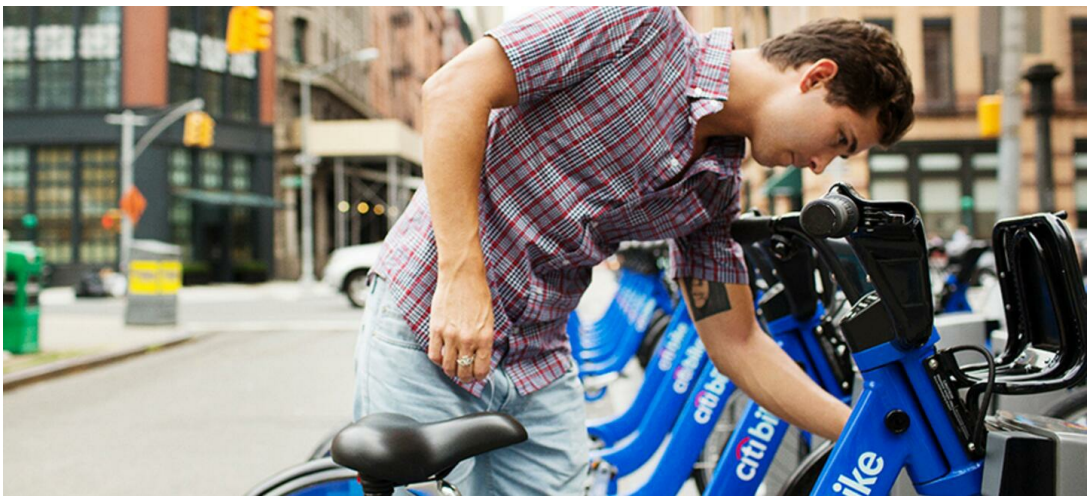
以下为计算用户会员类别和汇总饼图的代码。

```
#按用户的会员类别进行汇总并计算占比
user_type=cb.groupby('usertype')['bikeid'].agg(len)/cb["bikeid"].
count()*100

#汇总用户会员类别饼图
plt.rc('font', family='STXihei', size=15)
colors = ["#EA1F29","#39A2E1"]
name=['Customer', 'Subscriber']
plt.pie(user_type,labels=name,colors=colors,explode=(0,0),startan
gle=43,autopct='%1.1f%%')
plt.title('Citi Bike用户类别占比')
plt.legend(['Customer', 'Subscriber'], loc='upper left')
plt.show()
```

用户的骑行时间及速度

异步社区会员 lizhimiao(18926120246) 专享 尊重版权

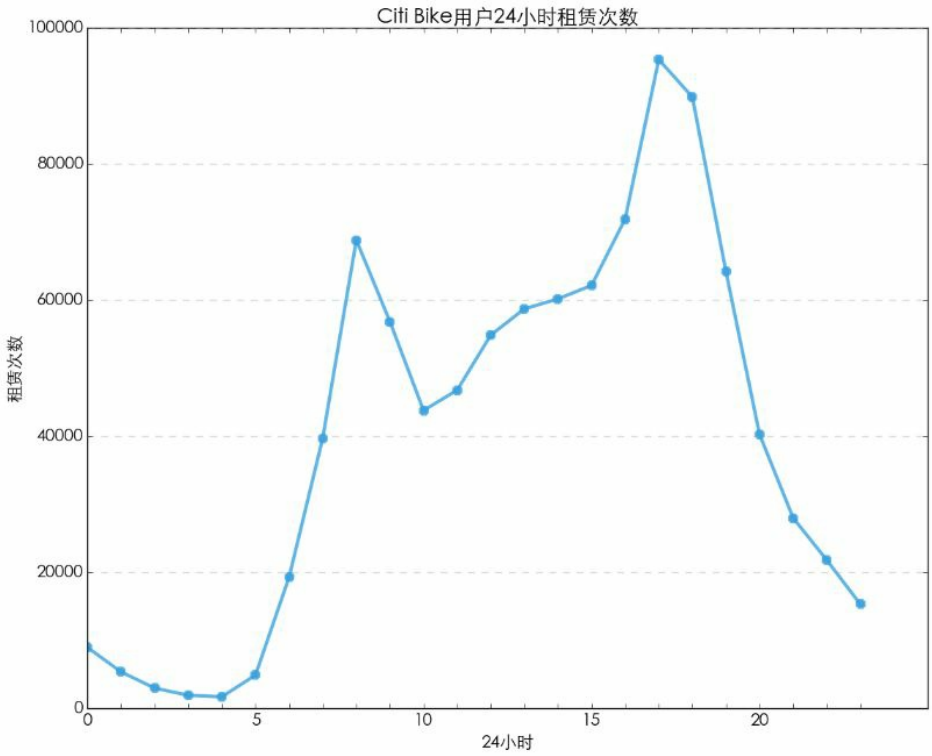


在993万次骑行的数据背后，是否存在一些规律？我们选择了5月(春季)的数据对用户使用Citi Bike的行为进行了统计和分析，这里既包括使用Citi Bike的时间，也包括骑行速度，热门租赁站点和骑行线路。

每日早晚是使用高峰

Citi Bike的使用者大部分为城市居民，少部分为游客。在一天中的上午7点—8点和下午的5点～6点是Citi Bike的使用高峰。这两个时间正好是上下班的高峰时间。除此之外中午12点～下午4点也有较高的使用量。

异步社区会员 lizhimiao(18926120246) 专享 尊重版权



以下是24小时使用趋势和绘制折线图的代码。

```
#读取5月数据并创建数据表
cb5=pd.DataFrame(pd.read_csv('201505-citibike-tripdata.csv'))
#对starttime字段进行分列
time_split = pd.DataFrame((x.split(' ') for x in cb5.starttime),i
ndex=cb5.index,columns=['start_date','star_time'])
#对分列后的表与原数据表进行拼接
cb5=pd.merge(cb5,time_split,right_index=True, left_index=True)
#更改star_time字段为日期格式
cb5['star_time']=pd.to_datetime(cb5['star_time'])
#设置star_time为表索引
cb5 = cb5.set_index('star_time')
#按小时对数据进行汇总
star_hour=cb5.resample('H',how=len)
#提取按小时汇总的bikeid数据
ride_hour=star_hour["bikeid"]

#绘制24小时折线图
plt.rc('font', family='STXihei', size=15)
```


异步社区会员 lizhimiao(18926120246) 专享 尊重版权

```
a=np.array([0,1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20,
21,22,23])
plt.plot(ride_hour,'8',ride_hour,'g-',color='#052B6C',linewidth=3
,markeredgewidth=3,markeredgecolor='#052B6C',alpha=0.8)
plt.xlabel('24小时')
plt.ylabel('租赁次数')
plt.title('Citi Bike用户24小时租赁次数')
plt.grid( color='#95a5a6',linestyle='--', linewidth=1 ,axis='y',a
lpha=0.4)
plt.xticks(a, ('0','','','','','5','','','','','10','','','','','
15','','','','','20','','','') )
plt.show()
```

骑行时间多在20分钟以内

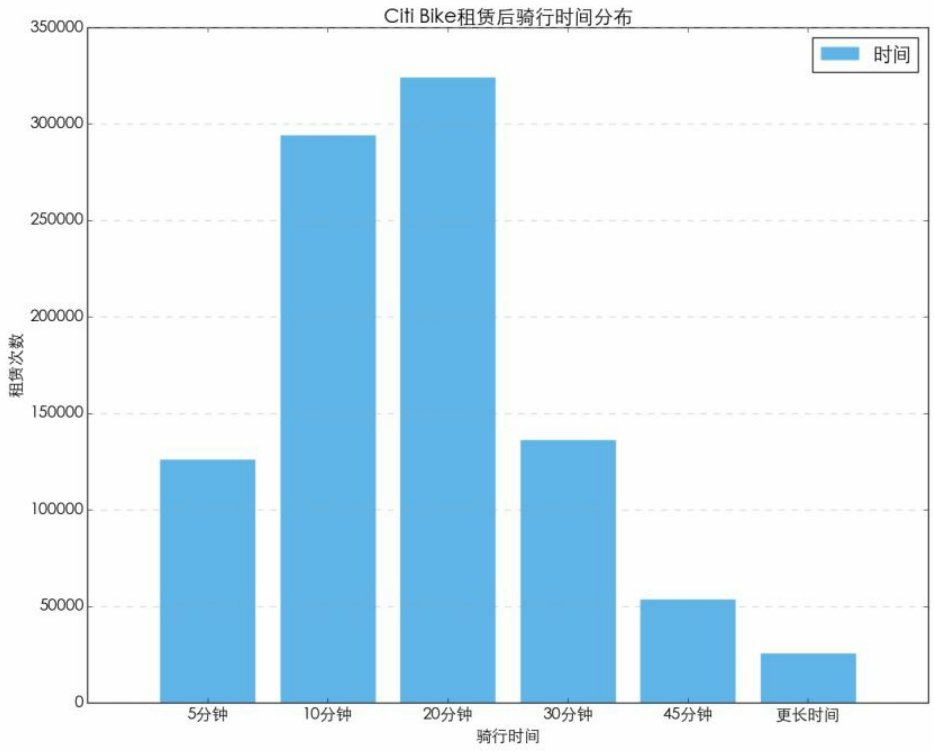
RIDE LENGTH	USAGE FEE
0-45 minutes	Included in membership
46–75 minutes	$2.50
76-105 minutes	+ $6.50
Each 30 min after 105 minutes	+ extra $9 / 30 min

在骑行时间方面，按照不同的会员类别Citi Bike对时间有不同的限制，1天和3天会员每次骑行的限制时间为30分钟，超过30分钟每增加15分钟收费4美金。年度会有每次骑行限制时间为45分钟，超出10分钟收费2.5美金，再次超出逐级收费。Citi Bike鼓励用户短途使用并且在接近限制时间时重新租赁新的自行车，避免产生超时费用，并且提高自行车的周转率。

我们对5月份用户的骑行时间进行处理和分组，通过下面的图表可以看出大部分用户的使用时间为10～20分钟。其次为30分钟和5分钟以内。超过30分钟的使用次数较少。即使在86%的用户为年度会员的情况下，使用时间在30-45分钟的情况也较少。由此推测大部分的

异步社区会员 lizhimiao(18926120246) 专享 尊重版权

骑行为短途。



以下为骑行时间分组和绘制柱状图的代码。

```
#对骑行时间进行分组
bins = [0, 300, 600, 1200, 1800, 2700, 2825827]
#为每个分组命名
group_tripduration = ['5分钟', '10分钟', '20分钟', '30分钟', '45分钟'
, '更长时间']
#在原数据表中增加时间分组字段
cb5['group_tripduration'] = pd.cut(cb5['tripduration'], bins, lab
els=group_tripduration)

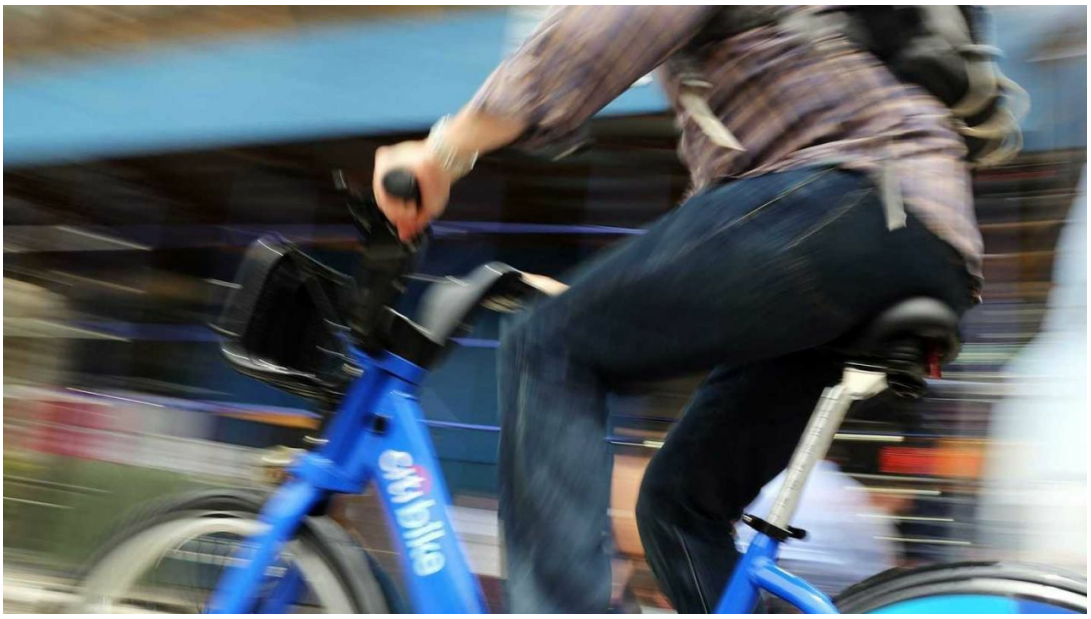
#按分组对数据进行汇总计数
group_minute=cb5.groupby('group_tripduration')['group_tripduratio
n'].agg(len)

#汇总骑行时间分组柱状图
plt.rc('font', family='STXihei', size=15)
```


异步社区会员 lizhimiao(18926120246) 专享 尊重版权

```
a=np.array([1,2,3,4,5,6])
plt.bar([1,2,3,4,5,6],group_minute,color='#9F713F',alpha=0.8,align='center',edgecolor='white')
plt.xlabel('时间分组')
plt.ylabel('骑行次数')
plt.title('Citi Bike骑行时间分布')
plt.legend(['时间'], loc='upper right')
plt.grid(color='#95a5a6',linestyle='--', linewidth=1,axis='y',alpha=0.4)
plt.xticks(a,('5分钟','10分钟','20分钟','30分钟', '45分钟','更长时间'))
plt.show()
```

平均骑行速度6.3公里/小时



在Citi Bike的数据表中并没有直接提供每次骑行的速度数据，但包含了每个开始和结束站点的经纬度坐标。我们通过这些坐标计算出了两个站点间的距离，并匹配到对应的骑行中。在经过与本次骑行所耗费的时间计算出这次骑行的平均速度。通过对5月的数据进行处理，用户的平均骑行速度为6.3公里/小时。

异步社区会员 lizhimiao(18926120246) 专享 尊重版权

以下是计算骑行速度的代码。

```
#导入库文件
from math import radians, cos, sin, asin, sqrt
import numpy as np
import pandas as pd

#通过经纬度计算距离的函数
def haversine(lon1, lat1, lon2, lat2): # 经度1，纬度1，经度2，纬度2（十进制度数）
    """
    Calculate the great circle distance between two points
    on the earth (specified in decimal degrees)
    """
    # 将十进制度数转化为弧度
    lon1= map(radians, np.array(lon1))
    lat1= map(radians, np.array(lat1))
    lon2= map(radians, np.array(lon2))
    lat2= map(radians, np.array(lat2))
    lon1 = np.array(list(lon1)).reshape(-1,1)
    lon2 = np.array(list(lon2)).reshape(-1,1)
    lat1 = np.array(list(lat1)).reshape(-1,1)
    lat2 = np.array(list(lat2)).reshape(-1,1)
    # haversine公式
    dlon = lon2 - lon1
    dlat = lat2 - lat1

    a = np.sin(dlat/2)**2 + np.cos(lat1) * np.cos(lat2) * np.sin(dlon/2)**2
    c = 2 * np.arcsin(np.sqrt(a))
    r = 6371 # 地球平均半径，单位为公里
    return c * r * 1000

#计算每次骑行的米数并增加骑行距离字段
cb5["meter"]=haversine(cb5["start station longitude"],cb5["start station latitude"],cb5["end station longitude"],cb5["end station latitude"])

#将原数据表中的骑行时间由秒转化为小时
cb5["duration_hour"]=cb5["tripduration"]/3600
#将米转化为公里并与小时计算出速度
cb5["speed"]=cb5["meter"]/1000/cb5["duration_hour"]

现在我们有了每次骑行的速度，还需要计算出用户平均的骑行速度。
#将每次骑行的米数求和并转化为公里
```


异步社区会员 lizhimiao(18926120246) 专享 尊重版权

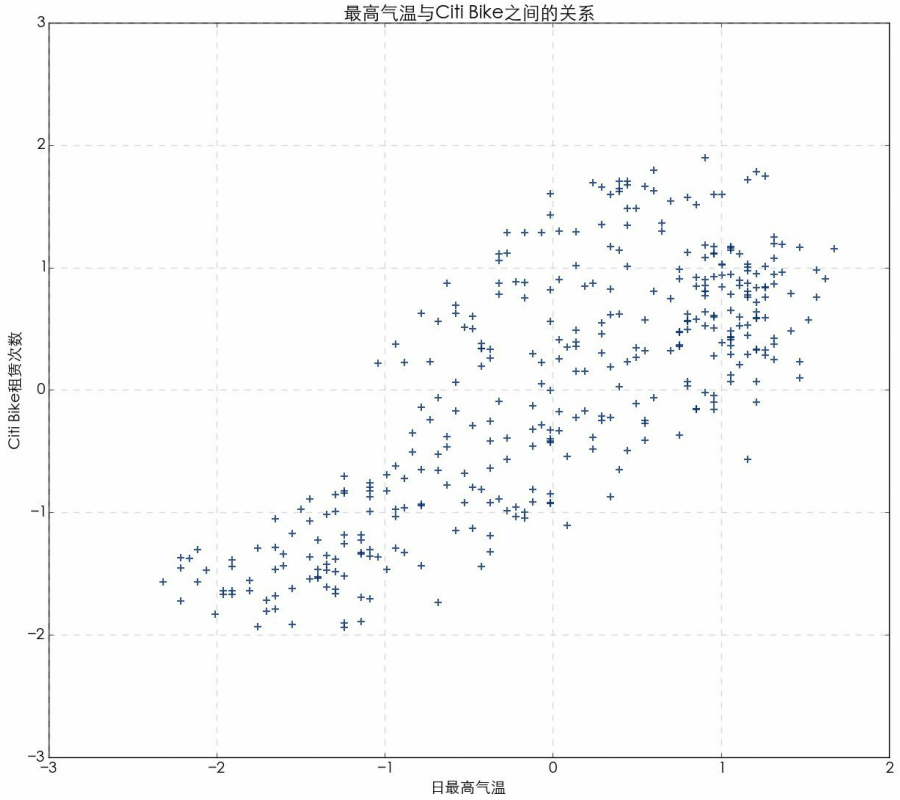
```
km=cb5["meter"].sum()/1000
#将每次骑行的秒数求和并转化为小时
hour=cb5["tripduration"].sum()/3600
#计算平均速度
speed=km/hour
#平均速度为6.31公里/小时
speed
6.3101247093495
```

骑行与气温是否存在关联？



在990万次骑行中，第一季度用户对Citi Bike的使用量最低，第三季度使用量最高。有明显的季节因素。下面我们导入纽约市2015年的气象数据，来看下天气因素与Citi Bike间是否存在关联，并试着用天气的变化来预测Citi Bike的使用量。

异步社区会员 lizhimiao(18926120246) 专享 尊重版权



绘制每日最高气温与Citi Bike使用量的散点图，从图中可以发现Citi Bike的使用量与日最高气温间呈正相关，随着日最高气温的增长Citi Bike的使用量也在逐渐增长。接下来我们通过回归分析发现最高气温与Citi Bike间的R平方值为0.6，两者存在正向关联。换句话说日最高气温可以解释Citi Bike租赁和骑行数量60%的变化原因。因此，我们通过这种关联分布对不同气温下的Citi Bike的租赁数量进行了预测。

以下是绘制散点图和进行回归分析的代码。

```
#读取2015年纽约市的气象数据
weather=pd.DataFrame(pd.read_csv('823248.csv'))
```


异步社区会员 lizhimiao(18926120246) 专享 尊重版权

```
#提取每日最高气温数据
group_weather_day=weather['TMAX']
#对2015年骑行时间按天汇总计算
cb_day=cb.resample('D',how=len)
#提起每日骑行数量
group_cb_day=cb_day['bikeid']

#对每日最高气温和骑行量数据进行标准化处理
from sklearn import preprocessing
scaler = preprocessing.StandardScaler().fit(group_weather_day)
X_Standard=scaler.transform(group_weather_day)
scaler = preprocessing.StandardScaler().fit(group_cb_day)
Y_Standard=scaler.transform(group_cb_day)

#绘制最高气温和骑行数据散点图
plt.rc('font', family='STXihei', size=15)
plt.scatter(X_Standard,Y_Standard,60,color='#052B6C',marker='+',a
lpha=0.8,linewidth=1.5)
plt.xlabel('日最高气温')
plt.ylabel('Citi Bike租赁次数')
plt.title('最高气温与Citi Bike之间的关系')
plt.grid(color='#95a5a6',linestyle='--', linewidth=1,axis='both',
alpha=0.4)
plt.show()

#设置每日最高气温为自变量X
X = np.array(weather[['TMAX']])
#设置每日骑行数量为因变量Y
Y = np.array(cb_day[['bikeid']])

#导入线性回归库
from sklearn import linear_model
clf = linear_model.LinearRegression()
clf.fit (X,Y)
#计算相关度
clf.score(X,Y)
0.60841860048565455
#斜率
clf.coef_
array([[ 536.64465091]])
#截距
clf.intercept_
array([-7282.61437278])
```

回归方程：

异步社区会员 lizhimiao(18926120246) 专享 尊重版权

Citi Bike租赁数量 = 536.64 × 最高气温 − 7282.61

```
#分布对19,39和97华氏度的Citi Bike租赁量进行预测
clf.predict(19)
array([[ 2913.63399443]])
clf.predict(39)
array([[ 13646.52701255]])
clf.predict(97)
array([[ 44771.9167651]])
```

骑行速度与年龄是否存在关联？



通过前面的计算我们已经知道了每次骑行用户的年龄和骑行速度信息，那么这两者之间是否存在关联呢？换句话说是否随着年龄的增长骑行速度会逐渐减慢呢？我们通过简单的一元回归来分析下两者间的联系。这里我们只使用2015年5月的数据进行分析。

```
#计算并增加年龄字段
cb5['age']=2015-cb5['birth year']
#提取年龄和骑行速度字段
age_speed=cb5[['speed','age']]
#去除骑行速度为0的数据
age_speed=age_speed.dropna()
#年龄设置为自变量X
X = np.array(age_speed[['age']])
#速度设置为因变量Y
```


异步社区会员 lizhimiao(18926120246) 专享 尊重版权

```
Y = np.array(age_speed[['speed']])

#导入线性回归库
from sklearn import linear_model
#将数据导入模型
clf = linear_model.LinearRegression()
clf.fit (X,Y)

#斜率
clf.coef_
array([[-0.02899706]])

#截距
clf.intercept_
array([ 10.05513438])

#R方
clf.score(X,Y)
0.011272777068773165
```

从R方来看年龄与骑行速度间并没有关联，骑行速度并不会随着年龄的增长而增长。这里主要的原因我想有两个。第一Citi Bike的使用场所主要在城市里，并且多为短途。第二使用者除了城市居民上下班通勤外，还有一部分的游客，他们的目的是欣赏沿途美丽的风景因此骑行速度也会较慢。

```
#20岁的骑行速度预测为9.4公里
clf.predict(20)
array([[ 9.47519327]])

#50岁的骑行速度预测为8.6公里
clf.predict(50)
array([[ 8.60528161]])
```

哪些骑行线路最受欢迎？

异步社区会员 lizhimiao(18926120246) 专享 尊重版权



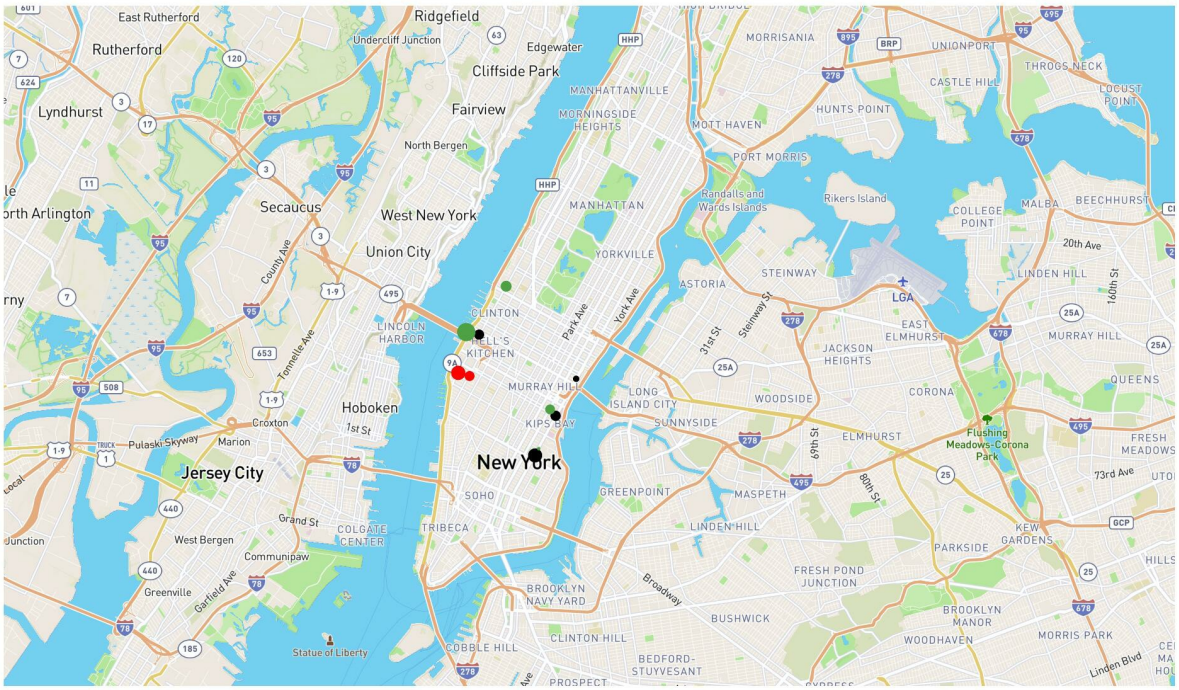
在Citi Bike的官网上有一个频道叫Explore NYC，里面提供了最热门的骑行线路和沿途的著名景观。我们对5月的数据进行统计，找出最受欢迎的骑行开始地点。以及受欢迎的骑行线路。

Top10受欢迎的租赁点

对5月骑行数据进行统计，找出前10个最受欢迎的骑行开始地点及经纬度数据。将经纬度数据输入到plotly工具中，绘制出Citi Bike 2015年5月在纽约最受欢迎的前10个租赁地点。

异步社区会员 lizhimiao(18926120246) 专享 尊重版权

Citi Bike 2015 年1月 TOP 10租赁地点



以下是前10个最受欢迎租赁点的统计代码及结果。

```
#使用数据透视找出前5个最受欢迎地点的经纬度数据
start_station=pd.pivot_table(cb5,index=["start station name","sta
rt station latitude","start station longitude"],values=['bikeid']
,aggfunc=[len],fill_value=0,margins=True).head(10)
```


异步社区会员 lizhimiao(18926120246) 专享 尊重版权

			len
			bikeid
start station name	start station latitude	start station longitude	
1 Ave & E 15 St	40.732219	-73.981656	4996
1 Ave & E 18 St	40.733812	-73.980544	1335
1 Ave & E 30 St	40.741444	-73.975361	3739
1 Ave & E 44 St	40.750020	-73.969053	2392
10 Ave & W 28 St	40.750664	-74.001768	3605
11 Ave & W 27 St	40.751396	-74.005226	5023
11 Ave & W 41 St	40.760301	-73.998842	3620
11 Ave & W 59 St	40.771522	-73.990541	3875
12 Ave & W 40 St	40.760875	-74.002777	6345
2 Ave & E 31 St	40.742909	-73.977061	3510

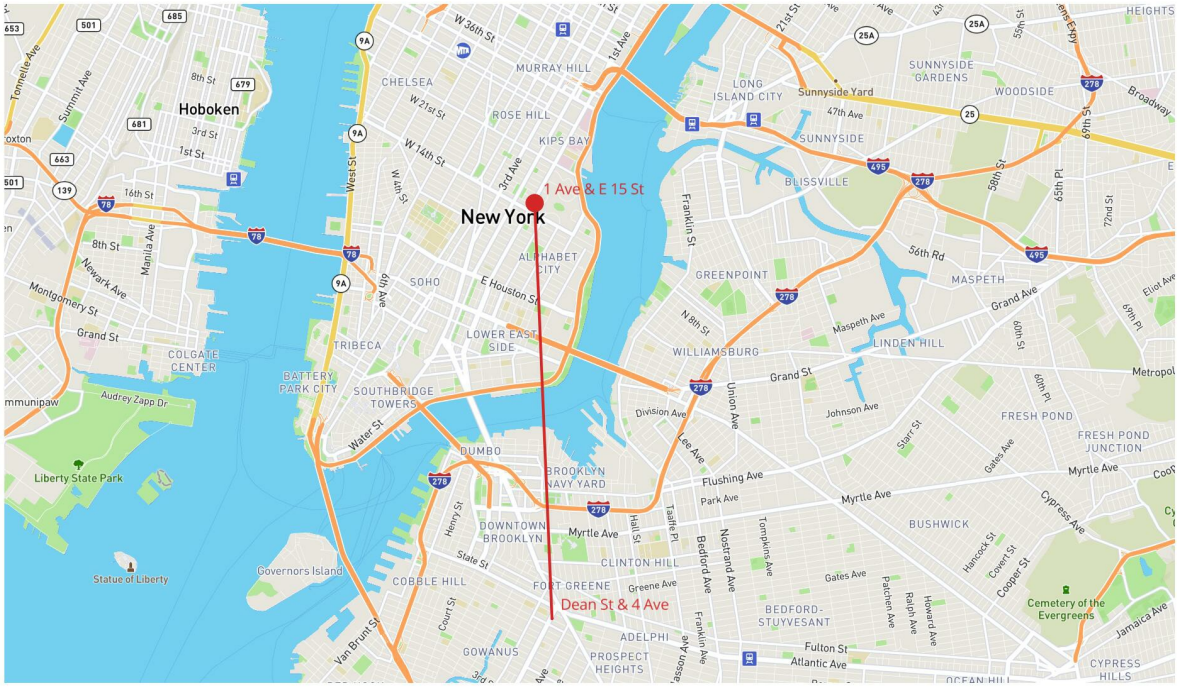
热门骑行线路追踪



继续前面的操作，在数据透视表的字段中增加结束地点和经纬度数据就可以看到一条完整的骑行线路数据，这里以1 Ave & E 15 St为骑行开始地点,使用plotly工具描绘出一条用户的骑行线路。上面的图中是从Google街景中截图的图片，也是骑行线路开始的地点。下面是plotly工具显示出的本次骑行开始和结束的位置。由于我们只有开始和结束两个经纬度数据，因此显示为一条直线。

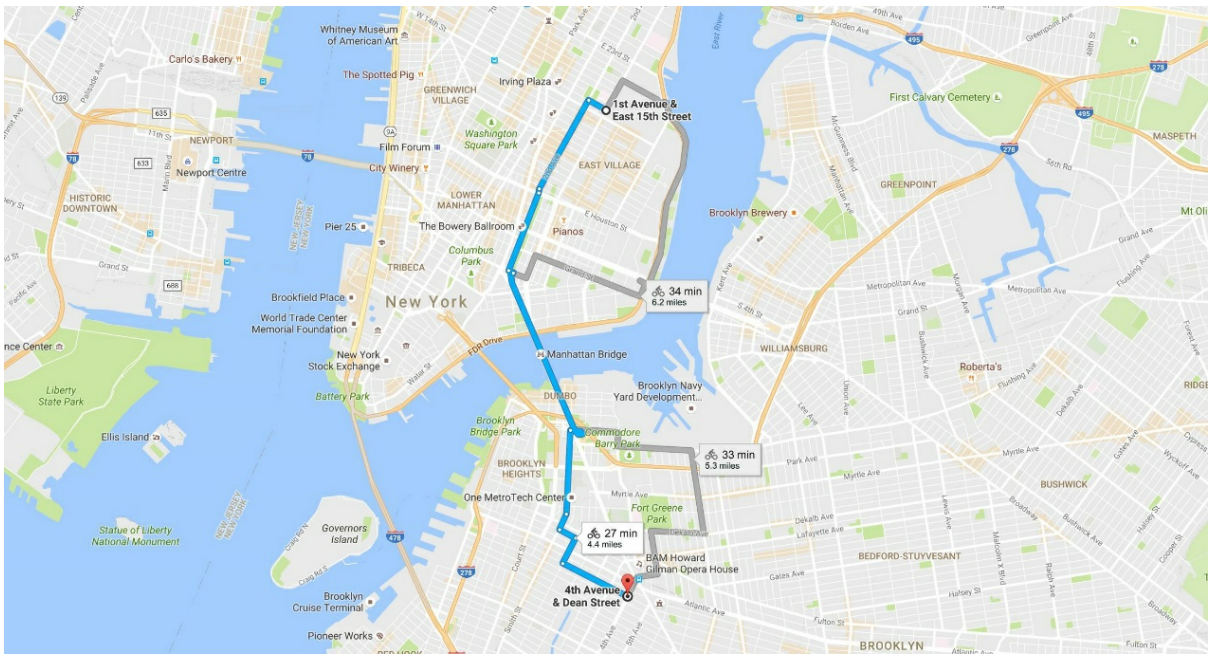
异步社区会员 lizhimiao(18926120246) 专享 尊重版权

Citi Bike 2015年1月热门骑行路径追踪



借助Google地图的骑行路线我们模拟出了用户这两点之间可能的骑行路线，这并非用户的真实骑行路线。但可以帮助我们更加详细的了解用户使用Citi Bike的情况。

异步社区会员 lizhimiao(18926120246) 专享 尊重版权



以下是通过数据透视获得骑行位置和经纬度的代码。

```
#获得骑行开始和结束地点及经纬度数据
end_station=pd.pivot_table(cb5,index=["start station name","start
 station latitude","start station longitude","end station latitud
e","end station longitude","end station name"],values=['bikeid'],
aggfunc=[len],fill_value=0,margins=True).head(10)
```

						len
						bikeid
start station name	start station latitude	start station longitude	end station latitude	end station longitude	end station name	
1 Ave & E 15 St	40.732219	-73.981656	40.683125	-73.978951	Dean St & 4 Ave	5
			40.683826	-73.976323	Atlantic Ave & Fort Greene Pl	1
			40.685282	-73.978058	Ashland Pl & Hanson Pl	3
			40.685396	-73.974315	S Portland Ave & Hanson Pl	1
			40.686919	-73.976682	Lafayette Ave & Fort Greene Pl	1
			40.687534	-73.972652	Cumberland St & Lafayette Ave	2
			40.688070	-73.984106	Bond St & Schermerhorn St	2
			40.688515	-73.964763	Lafayette Ave & St James Pl	1
			40.689269	-73.989129	State St & Smith St	2
			40.690893	-73.996123	Henry St & Atlantic Ave	1

异步社区会员 lizhimiao(18926120246) 专享 尊重版权

结束语

我们使用Python对纽约自行车共享系统Citi Bike 990万次骑行数据的简单分析，在惊叹于用户对自行车共享系统的热爱和使用频率的同时，也学习到很多用户骑行的信息。使用Citi Bike的用户以纽约市的中年男性为主，在每天的早晨的8点和傍晚6点是Citi Bike的使用高峰。由于Citi Bike对超时单独收取费用，用户的骑行时间多数在20分钟以内。用户对Citi Bike的使用受季节和气温的影响，相关度达到0.6，夏秋两季租赁量最大，冬季最低。用户的年龄与骑行速度关联并不紧密，平均骑行速度为6.3公里/小时。

以上是我们通过本次分析获得的结论，但对于Citi Bike和用户骑行的分析并没有结束。后续我们还会对在博客和公众号对Citi Bike的数据进行更多专项的分析和对比，如用户在每周的骑行趋势是什么样的？工作日和休息日Citi Bike的使用率是否不同？一辆自行车在一天中的路径是什么样的？除了气温以外，还有哪些因素影响了用户对Citi Bike的使用？敬请期待。

异步社区会员 lizhimiao(18926120246) 专享 尊重版权